REPUBLIC OF TUNISIA
MINISTRY OF HIGHER EDUCATION
AND SCIENTIFIC RESEARCH
UNIVERSITY OF SFAX




# GRADUATION PROJECT

FOR THE OBTAINMENT OF THE BACHELOR'S DEGREE IN

**COMPUTER SCIENCE APPLIED TO MANAGEMENT**

**A Web & Mobile Platform for Kindergarten Management and Child Monitoring**

**PREPARED BY:** Mahmoud Kammoun

**ACADEMIC SUPERVISOR:** Dr. Marwa Graja

**PROFESSIONAL SUPERVISOR:** Mr. Mohamed Amine Ben Ayed

**JURY MEMBERS:**

Dr. Monia Ben Brahim

Dr. Emna Feki

**INTERNSHIP PERFORMED AT: Mega Pixel**

**Academic Year 2025/2026**

# Acknowledgments

I would like to express my sincere gratitude to all those who contributed to the completion of this work.

I would especially like to thank **Mrs. Marwa Graja** for her rigorous supervision, valuable guidance, and continuous support throughout this project.

I would also like to extend my thanks to **Mr. Mohamed Amine Ben Ayed**, CEO of Mega Pixel, for his support, guidance, and availability during the practical phase of this work, and to **Mrs. Yasmine Tounsi** for her help and support during this work.

My appreciation also goes to the teaching staff of the **Faculty of Economics and Management of Sfax (FSEGS)**, as well as to all the collaborators at Mega Pixel, for their warm welcome and valuable assistance.

I would also like to express my gratitude to the members of the jury, **Mrs. Monia Ben Brahim** and **Mrs. Emna Feki**, for the attention and interest they have given to this thesis.

Finally, I would like to express my deepest thanks to everyone who supported and encouraged me, directly or indirectly, in the completion of this work.

# Dedications

*To those who made this journey more than just an academic path.*

**To my parents,**

You have been my constant in uncertainty, my strength in doubt,
and my guide at every step along the way.
This work silently carries the mark of everything you have given me.

**To my brother and my sister,**

For your quiet yet essential presence, your encouragement,
and the energy that has always pushed me forward.

**To my friends,**

Who turned ordinary moments into precious memories
and who were there in both seriousness and laughter.

**And to all those who, near or far, believed in me,**
This work is yours as well.

# Contents


**General Introduction** 10

**1 Project Definition and Analysis of Existing Solutions** 11
- **1.1 Introduction** 11
- **1.2 Mission Definition** 11
  - **1.2.1 About the Host Organization** 12
  - **1.2.2 Application Overview** 12
  - **1.2.3 Objectives to be Achieved** 14
- **1.3 Project Context** 14
- **1.4 Analysis of Existing Applications** 14
  - **1.4.1 Brightwheel Application** 15
  - **1.4.2 Kinderpedia Application** 16
  - **1.4.3 Synthesis and Comparison** 16
- **1.5 Problem Statement and Proposed Solution** 17
- **1.6 Conclusion** 17

**2 Requirements Gathering** 18
- **2.1 Introduction** 18
- **2.2 Identification of System Actors** 18
- **2.3 Context of the Information System** 19
- **2.4 Identification of Use Cases** 20
  - **2.4.1 Use Case Diagram** 20
- **2.5 Textual Description of Use Cases** 21
  - **2.5.1 1. Parent Use Cases** 21
  - **2.5.2 2. Teacher Use Cases** 26
  - **2.5.3 3. Teacher and Parent Use Cases** 29
  - **2.5.4 4. Admin Use Cases** 31
  - **2.5.5 5. All Actors Use Cases** 46
- **Conclusion** 47

**3 Analysis and Design** 48
- **3.1 Introduction** 48
- **3.2 Realization of Use Cases** 48
  - **3.2.1 Parent: Track Children - Scheduled Activities** 49
  - **3.2.2 Parent: Track Children - Attendance Status** 49
  - **3.2.3 Parent: Track Children - Acquired Competences** 50
  - **3.2.4 Parent: View Photos** 51
  - **3.2.5 Parent: Add Child - Register a new child with medical and registration information** 52
  - **3.2.6 Parent: Register Parent - Submit Registration Form** 53
  - **3.2.7 Admin: Manage Registrations - Approve a Pending Registration Request** 55
- **3.3 Domain Model Construction (Class Diagram)** 57

3.3.1 Detailed Class Definitions . . . 58
3.4 Future Software Architecture . . . 63
3.5 Physical Data Schemas . . . 64
3.6 Conclusion . . . 65

4 Implementation 66
4.1 Introduction . . . 66
4.2 Development Environment . . . 66
4.2.1 Hardware Environment . . . 66
4.2.2 Software Environment and Technologies . . . 66
4.2.3 Deployment and Infrastructure Tools . . . 67
4.3 Technical Implementation of Use Cases . . . 69
4.3.1 Parent Use Cases . . . 69
4.3.2 Teacher Use Cases . . . 73
4.3.3 Parent and Teacher Shared Use Cases . . . 77
4.3.4 Administrator Use Cases . . . 79
4.3.5 System API Endpoints . . . 87
4.3.6 Database Triggers . . . 89
4.4 Conclusion . . . 90

General Conclusion 91

Bibliography 92

Webography 93

# List of Figures


1.1 Mega Pixel Logo . . . 12
1.2 Graphical interface of the Brightwheel platform . . . 15
1.3 Graphical interface of the Kinderpedia platform . . . 16

2.1 Context Diagram of the Information System . . . 19
2.2 Global Use Case Diagram of the Petit Suivi System . . . 20

3.1 Sequence Diagram: Consult today's scheduled activities . . . 49
3.2 Sequence Diagram: Check daily attendance status . . . 49
3.3 Sequence Diagram: Review acquired competences . . . 50
3.4 Class Diagram from UC: Track Children . . . 50
3.5 Sequence Diagram: Browse photo gallery . . . 51
3.6 Class Diagram from UC: View Photos . . . 51
3.7 Class Diagram: Add Child and Enrollment . . . 52
3.8 Class Diagram: Create a New Parent Account . . . 53
3.9 Class Diagram: Register Parent . . . 54
3.10 Sequence Diagram: Approve a pending registration request . . . 55
3.11 Class Diagram from UC: Manage Registrations . . . 56
3.12 Global Class Diagram for Petit Suivi . . . 57
3.13 Future Software Architecture . . . 63
3.14 Physical Data Schemas . . . 64

4.1 CI/CD Pipeline Configuration . . . 67
4.2 Cloudflare Tunnel Architecture . . . 68
4.3 Graphical interface of the track children dashboard . . . 69
4.4 Graphical interface for viewing today's scheduled activities . . . 69
4.5 Graphical interface for viewing daily attendance . . . 70
4.6 Graphical interface for viewing acquired competences . . . 70
4.7 Graphical interface for browsing the photo gallery . . . 71
4.8 Graphical interface for child re-registration . . . 71
4.9 Graphical interface for viewing payment records . . . 72
4.10 Graphical interface for viewing class activities . . . 73
4.11 Graphical interface for photo sharing and upload . . . 73
4.12 Graphical interface for completing the photo sharing workflow . . . 74
4.13 Graphical Interface for viewing assigned classes . . . 75
4.14 Graphical interface for recording class attendance . . . 75
4.15 Graphical interface for evaluating competences . . . 76
4.16 Graphical interface for viewing parent profile . . . 77
4.17 Graphical interface for viewing teacher profile . . . 77
4.18 Graphical interface for viewing notifications . . . 78
4.19 Graphical interface for mobile login . . . 78
4.20 Graphical interface for viewing dashboard statistics . . . 79
4.21 Graphical interface for activity creation . . . 79
4.22 Graphical interface for viewing payments . . . 80
4.23 Graphical interface for viewing pending registrations . . . 80

4.24 Graphical interface for approving registrations . . . . . 81
4.25 Graphical interface for creating classes . . . . . 81
4.26 Graphical interface for approving parent accounts . . . . . 82
4.27 Graphical interface for viewing users . . . . . 82
4.28 Graphical interface for meal planning . . . . . 83
4.29 Graphical interface for viewing planned activities . . . . . 83
4.30 Implementation code: validate planned activity assignment rules . . . . . 84
4.31 Implementation code: detect planned activity scheduling conflicts . . . . . 84
4.32 Graphical interface for viewing system parameters . . . . . 85
4.33 Graphical interface for viewing system parameters . . . . . 85
4.34 Graphical interface for viewing financial reports . . . . . 86
4.35 Graphical interface for viewing financial reports . . . . . 86

# List of Tables


2.1 Textual Description of Use Case 1.1 – Track Children . . . . . 22
2.2 Textual Description of Use Case 1.2 – Review Payment History . . . . . 23
2.3 Textual Description of Use Case 1.3 – View Photo . . . . . 23
2.4 Textual Description of Use Case 1.4 – Register Parent . . . . . 24
2.5 Textual Description of Use Case 1.5 – Add Child . . . . . 25
2.6 Textual Description of Use Case 2.1 – Evaluate Child Competences and Behavior 26
2.7 Textual Description of Use Case 2.2 – Manage Class Attendance . . . . . 27
2.8 Textual Description of Use Case 2.3 – View and Edit Activities . . . . . 28
2.9 Textual Description of Use Case 2.4 – Send Photos . . . . . 28
2.10 Textual Description of Use Case 3.1 – Contact and Support . . . . . 29
2.11 Textual Description of Use Case 3.2 – Manage Profile . . . . . 31
2.12 Textual Description of Use Case 3.3 – View Notifications . . . . . 31
2.13 Textual Description of Use Case 4.1 – Access Dashboard . . . . . 31
2.14 Textual Description of Use Case 4.2 – Manage Users (Parent, Teacher) . . . . . 33
2.15 Textual Description of Use Case 4.3 – Manage Classes . . . . . 34
2.16 Textual Description of Use Case 4.4 – Manage Registrations . . . . . 35
2.17 Textual Description of Use Case 4.5 – Manage Payments . . . . . 36
2.18 Textual Description of Use Case 4.6 – Manage Events . . . . . 37
2.19 Textual Description of Use Case 4.7 – Manage Activities . . . . . 39
2.20 Textual Description of Use Case 4.8 – Manage Planned Activities . . . . . 40
2.21 Textual Description of Use Case 4.9 – Manage Criteria . . . . . 41
2.22 Textual Description of Use Case 4.10 – Plan Meal Schedules . . . . . 42
2.23 Textual Description of Use Case 4.11 – Manage Meals . . . . . 43
2.24 Textual Description of Use Case 4.12 – Manage Food Exceptions . . . . . 44
2.25 Textual Description of Use Case 4.13 – Generate Behavioral Reports . . . . . 45
2.26 Textual Description of Use Case 4.14 – Generate Payment Reports . . . . . 45
2.27 Textual Description of Use Case 4.15 – Configure Params . . . . . 46
2.28 Textual Description of Use Case 5.1 – Authenticate . . . . . 47

3.1 Class definition: Role . . . . . 58
3.2 Class definition: Account . . . . . 58
3.3 Class definition: Notification . . . . . 58
3.4 Class definition: NotificationType . . . . . 58
3.5 Class definition: Event . . . . . 58
3.6 Class definition: Parameter . . . . . 58
3.7 Class definition: DietaryComment . . . . . 58
3.8 Class definition: MedicalForm . . . . . 58
3.9 Class definition: HealthComment . . . . . 59
3.10 Class definition: PartialPayment . . . . . 59
3.11 Class definition: Payment . . . . . 59
3.12 Class definition: Registration . . . . . 59
3.13 Class definition: PaymentMethod . . . . . 59
3.14 Class definition: RegistrationType . . . . . 59
3.15 Class definition: MealPlan . . . . . 59

3.16 Class definition: RegistrationStatus . . . 59
3.17 Class definition: PresenceStatus . . . 59
3.18 Class definition: Presence . . . 60
3.19 Class definition: ChildPhoto . . . 60
3.20 Class definition: Child . . . 60
3.21 Class definition: ChildPhotoRecipient . . . 60
3.22 Class definition: PickupNotification . . . 60
3.23 Class definition: PickupNotificationsStatus . . . 60
3.24 Class definition: Class . . . 60
3.25 Class definition: ClassType . . . 60
3.26 Class definition: ChildFoodException . . . 60
3.27 Class definition: DailyPlan . . . 60
3.28 Class definition: MenuPlanning . . . 60
3.29 Class definition: Meal . . . 61
3.30 Class definition: MealCategory . . . 61
3.31 Class definition: ChildFoodExceptionOverride . . . 61
3.32 Class definition: SchoolYear . . . 61
3.33 Class definition: PlanDay . . . 61
3.34 Class definition: PlanDayActivity . . . 61
3.35 Class definition: ActivityStatus . . . 61
3.36 Class definition: Activity . . . 61
3.37 Class definition: Criteria . . . 61
3.38 Class definition: Evaluation . . . 61
3.39 Class definition: Grade . . . 62
3.40 Class definition: GradeStatus . . . 62
3.41 Class definition: Signal . . . 62
3.42 Class definition: ReportAnalysisHistory . . . 62

4.1 System API Endpoints - Part 1 . . . 87
4.2 System API Endpoints - Part 2 . . . 88

# General Introduction

In any educational institution, the Information System aims to facilitate the establishment and implementation of pedagogical and administrative management strategies. It must support the execution of daily activities such as enrollment management and activity monitoring while ensuring an efficient flow of information between parents, teachers, and the administration.

The preschool institution's information system is designed to meet professional requirements and consists of a set of hardware and software resources used to collect, store, and process data related to children's development. One of the greatest assets of a preschool institution lies in this information, which is often heterogeneous, concerning the medical, educational, and behavioral monitoring of each child.

Despite this wealth of information, the absence of an integrated computerized tool makes it difficult to obtain a comprehensive and coherent overview of student progress and administrative management. Furthermore, manual access to information stored across different media, including paper files, allergy lists, and attendance registers, makes data handling complex and inefficient.

It is within this context that we examine the case of preschool management. Given the significant volume and sensitivity of the information handled in these institutions, we were tasked with automating their Information System as part of a final year project entitled Petit Suivi.

The objective of this project is to develop a comprehensive management platform that facilitates the efficient use of service resources, such as teachers, classes, and meal plans, to carry out daily tasks. The project also aims to strengthen communication between parents and the school, ensure accurate tracking of each child's development, and optimize work organization through innovative tools, particularly artificial intelligence for behavioral monitoring.

To accomplish the required work, we adopted a design methodology based on the Rational Unified Process and the Unified Modeling Language. The results of our design work are presented in this report, which is structured into four distinct chapters.

In the first chapter, Business Modeling, we analyze the existing system by highlighting the objectives to be achieved. We define the scope of the study and examine in detail the business processes within the educational institution. The second chapter, Requirements Gathering, focuses on identifying the actors of the future application, including administrators, teachers, and parents, while developing the context model and describing the use case model. In the third chapter, we analyze user requirements, design use case scenarios, and define the application classes, the system architecture, and the physical database schema. Finally, the fourth chapter, Implementation, describes the development environment involving Laravel, Flutter, and React, presents the technical realization of each use case with interface screenshots, and documents the system API endpoints.

This report concludes with a general summary of the work accomplished and proposes future improvements to further enhance the application.

# Chapter 1

# Project Definition and Analysis of Existing Solutions

## 1.1 Introduction

Business modeling is an effective way to manage the complexity of a project and ensure its overall consistency. This phase is essential to achieve a shared understanding among stakeholders and to clearly define the problem to be addressed.
This stage begins with an analysis of the existing system and a precise definition of our mission. It also involves identifying the project scope, describing business processes, and determining the information flows involved.

## 1.2 Mission Definition

Our mission within the framework of this project is to design and develop a comprehensive platform, along with its mobile application, aimed at facilitating the daily management of preschool institutions.

This solution enables administrators, teachers, and parents to efficiently manage:

- Registration processes and user account management
- Financial monitoring and monthly payment tracking
- Child monitoring and pedagogical competence evaluations
- Activity scheduling, event management, and meal planning (including dietary restrictions)
- Real-time notifications and communication between stakeholders

The following subsections present the host organization, the application, and the objectives to be achieved.

### 1.2.1 About the Host Organization

Mega Pixel is an innovative digital agency specialized in creating customized digital solutions. It was founded with the aim of supporting companies in their digital transformation.
Mega Pixel brings together a team of passionate experts in the fields of web development, digital marketing, graphic design, video production, and social media sponsorship.
To address the growing needs in early childhood education, including registration management, precise child monitoring, financial tracking, activity coordination, and instant internal communication, Mega Pixel has conceptualized and developed the **Petit Suivi** project.




**Figure 1.1:** Mega Pixel Logo

### 1.2.2 Application Overview

The **Petit Suivi** application is a kindergarten management platform dedicated to preschool institutions.
It addresses three major needs:

**For Administration:**

- Centralized management of users, classes, and registrations
- Payment tracking and registration fees management
- Configuration of system parameters and global contacts
- Meal orchestration and detailed student allergy handling
- Custom event planning and validation
- Activity scheduling and competence criterion management
- Advanced reporting and financial data visualization

**For Teachers:**

- Class and child attendance tracking
- Reviewing planned activities for their assigned classes
- Evaluating child competences and submitting behavioral reports
- Capturing and sending photos to parents
- Viewing notifications

**For Parents:**

- Submitting child registrations and updating medical information
- Monitoring child attendance, activities, and competence evaluations
- Viewing and downloading shared photos
- Reviewing monthly payment history and receipts
- Receiving real-time system notifications
- Managing profiles and accessing support services

The application is structured into two complementary components:

### 1. Mobile Component

The mobile application primarily targets parents and teachers.
Main features include:

- Authentication (login/registration) with validation workflow
- Role-based dashboard (parent/teacher)
- Child monitoring (attendance, activities, specific medical alerts)
- Teacher features (class attendance management, competence evaluations, reporting)
- Parent features (payment history, notifications, profile management )
- Real-time photo gallery sharing
- Support and contact interface

### 2. Web Administration Component

The administrative web platform includes:

- **Dashboard:** global indicators and operational statistics
- **User Management:** administration of teacher and parent accounts
- **System Settings:** global configurations, fees, and contacts
- **Class Management:** levels, capacities, and teacher assignments
- **Registration Management:** request validation, filtering, and assignment
- **Payment Management:** tracking transactions, monthly fees, and generating receipts
- **Activity & Event Management:** planning, calendar assignment, and execution tracking
- **Meal Orchestration:** weekly planning, catalog updates, and allergy handling
- **Reporting:** financial indicators, payment monitoring, and dietary exception tracking

### 1.2.3 Objectives to be Achieved

The Petit Suivi application aims to:

- Centralize the management of registrations, payments, classes, activities, and meals
- Ensure child safety through strict tracking of medical / dietary needs (allergies) and real-time attendance
- Reduce manual administrative tasks and parameter configurations
- Streamline daily operations and financial reporting
- Improve pedagogical monitoring through precise competence evaluations
- Strengthen communication between administration, staff, and families via photo sharing and notifications
- Provide reliable indicators and comprehensive dashboards for optimal decision-making

## 1.3 Project Context

The management of educational institutions, particularly in early childhood, entails complex administrative and pedagogical challenges. Currently, many local preschools rely on traditional, paper-based methods or disjointed digital tools (such as separate messaging groups, spreadsheets, and paper notebooks). This lack of centralization often leads to inefficiencies: high administrative burdens, fragmented parent-teacher communication, difficulties tracking payments, and challenges in maintaining comprehensive records of child development.
In this context, there is a growing need for digital transformation. Institutions require comprehensive solutions that bring administrators, teachers, and parents onto a single, intuitive platform to streamline operations and enhance the overall educational experience.

## 1.4 Analysis of Existing Applications

This section presents a comparative study of similar applications already available on the market. The objective is to analyze existing solutions in order to understand their strengths and weaknesses, identify best practices, and highlight the added value that the Petit Suivi application will provide.
Studying existing platforms allows us to:

- Identify commonly implemented features.
- Detect functional or technical limitations.
- Evaluate usability and ergonomics.
- Determine opportunities for innovation and improvement.

Two major applications have been selected for analysis: Brightwheel and Kinderpedia, both widely used in preschool and early childhood education management.

### 1.4.1 Brightwheel Application

Brightwheel is one of the leading childcare management software platforms, originally developed in the United States. It offers an all-in-one platform covering attendance tracking, billing, staff management, and parent communication. This solution proposes several advanced features:

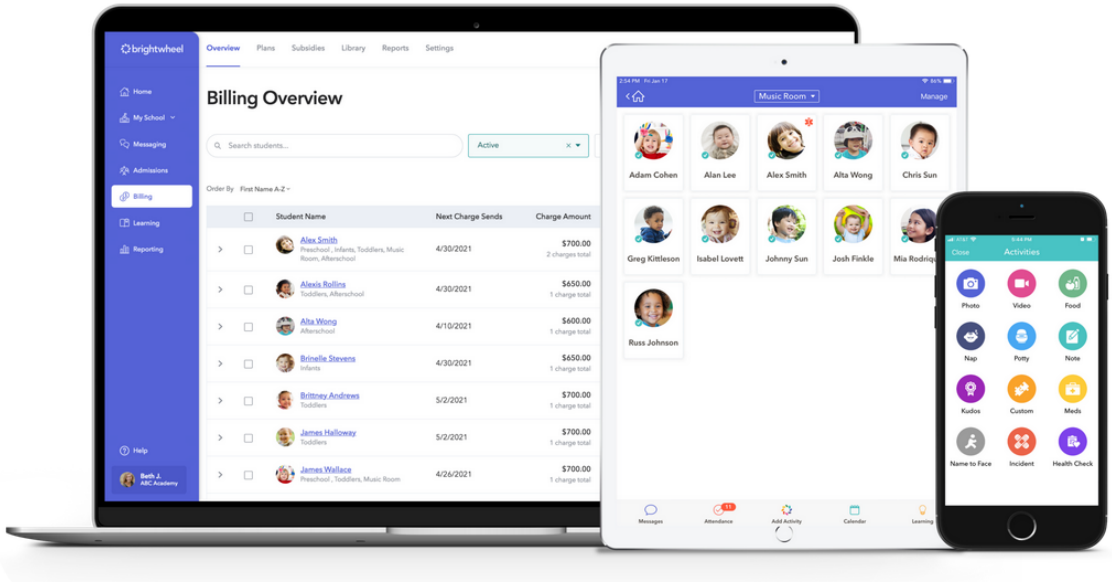



**Figure 1.2:** Graphical interface of the Brightwheel platform

- **Comprehensive Features:** Includes robust billing and automated invoicing tools to streamline financial management.
- **Real-Time Updates:** Enables excellent real-time photo and video sharing, ensuring parents stay connected.
- **Ease of Use:** Provides a very intuitive mobile interface tailored specifically for teachers' daily tasks.

Although Brightwheel offers a complete range of features adapted to early childhood management, several factors limit its direct adoption for our host institution:

- **Cost limitations:** High recurring subscription costs designed for international markets represent a significant, ongoing financial burden that can be prohibitive for local nurseries.
- **Customization limitations:** The platform is difficult to adapt to specific local administrative procedures, unique class structures, and targeted pedagogical competency frameworks required by our curriculum.
- **Data Control limitations:** Relying on external, international servers implies less control over sensitive private data and documents, a crucial aspect for the institution.

### 1.4.2 Kinderpedia Application

Kinderpedia is an award-winning European school management software focusing heavily on family engagement, automated invoicing, and digital class management. It stands out by providing deeply integrated communication modules with several key features:

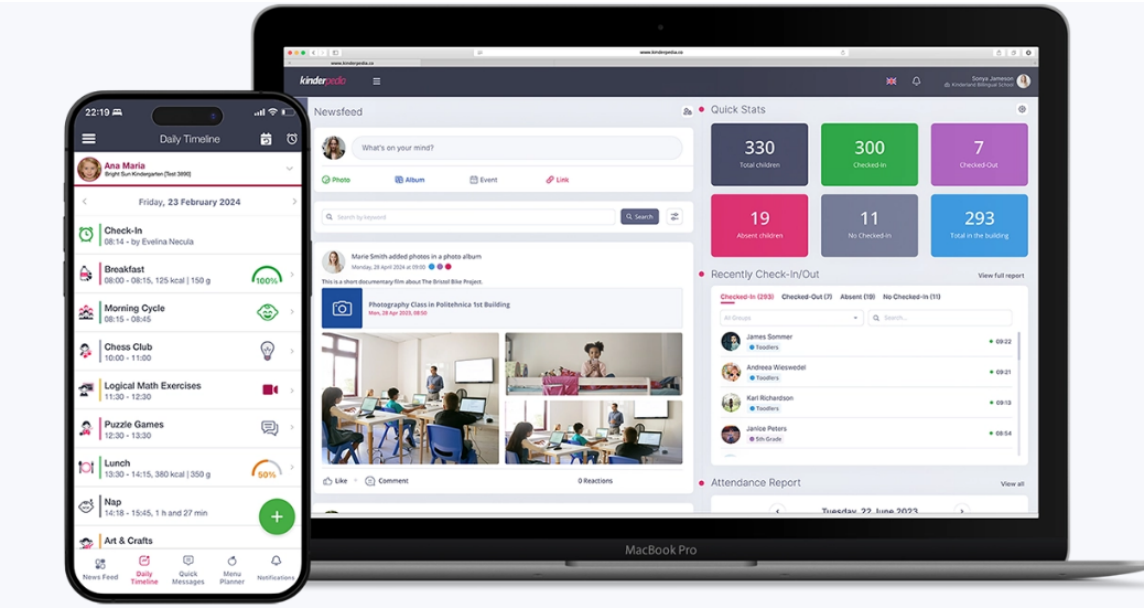



**Figure 1.3:** Graphical interface of the Kinderpedia platform

- **Multi-language Support:** Offers an interface available in several languages, making it accessible to diverse staff and parents.
- **Family Engagement:** Features dedicated tools designed to keep parents heavily involved in tracking a child's academic and behavioral progress.
- **Automated Reports:** Generates comprehensive and excellent attendance and financial reports for administrators.

While Kinderpedia is highly regarded in the European market, certain aspects restrict its suitability for our specific needs:

- **Platform Complexity:** The abundance of advanced modules makes the platform somewhat bloated for small-to-medium nurseries, resulting in a steep learning curve and overly complex onboarding for staff.
- **Fixed Workflows:** It lacks the precise flexibility required to fully customize meal tracking processes and detailed behavioral evaluations according to purely local operational standards.
- **Pricing Model:** The structure requires expensive premium tiers to access the necessary reporting functionalities.

### 1.4.3 Synthesis and Comparison

Based on the analysis of these platforms, both Brightwheel and Kinderpedia provide strong communication, attendance, and administrative features. However, they remain

limited in terms of local customization, deployment cost, and full control over institutional data, which reinforces the relevance of developing Petit Suivi as a solution tailored to the host institution.

## 1.5 Problem Statement and Proposed Solution

The critical analysis of the existing systems highlights several limitations. While solutions like Brightwheel and Kinderpedia offer rich functionalities, they rely on expensive, recurring subscription models targeted at European and North American markets. Furthermore, these platforms lack the flexibility to adapt to the specific administrative workflows, customized meal planning, and localized pedagogical tracking methods required by our institution. Finally, relying on third-party international cloud solutions raises concerns regarding long-term data ownership and localization.
**Proposed Solution:** To address these challenges, we propose the development of **Petit Suivi**. This custom-tailored solution will provide a robust, unified platform consisting of a web-based administration panel and a dedicated mobile application for parents and teachers. By developing this system in-house, the institution will benefit from:

- **Total Ownership:** Elimination of third-party licensing fees and full control over data hosting and security.
- **Tailored Functionality:** Workflows specifically designed around the institution's actual operations, from customized registration forms to local competency tracking.
- **Centralized Communication:** A seamless digital bridge replacing fragmented communication channels, offering real-time notifications, absence alerts, and photo sharing.

## 1.6 Conclusion

In this chapter, we established the general framework of the project. We clearly defined the mission, identified the primary actors, and outlined the system's core objectives. Through a critical study of existing applications, we gathered valuable insights into the market's current capabilities and limitations. These conclusions justify our decision to develop a fully custom solution. With the context and requirements clearly defined, the next chapter will focus on the detailed functional requirements and the specifications of the Petit Suivi system.

# Chapter 2

# Requirements Gathering

## 2.1 Introduction

This chapter focuses on the requirements gathering phase, which is a fundamental step in any software development process. By analyzing the needs of all stakeholders, we define the functional scope of the **Petit Suivi** system.
We begin by identifying the key actors who interact with the platform, then model the information system context. Next, we identify and classify all use cases using a UML use case diagram, and finally provide detailed textual descriptions for each use case scenario.

## 2.2 Identification of System Actors

An actor represents any external entity whether a person, a device, or another system that interacts with the system under study. The Petit Suivi platform identifies three primary actors:

- **Parent:** Accesses the system through the mobile application. Parents can register themselves and their children, track child activities, attendance, and competence evaluations, view photos, manage their profile, consult payment history, receive notifications, and contact nursery support.
- **Teacher:** Accesses the system through the mobile application. Teachers can view planned activities for their assigned class, manage class attendance, evaluate child competences, send photos to parents, manage their profile, view notifications, and contact support.
- **Administrator (Admin):** Accesses the system through the web administration panel. The admin manages all aspects of the nursery including: user accounts (parents and teachers), class management, registration approvals, payment tracking, activity and competence management, meal planning, event management, system configuration, and report generation.

## 2.3 Context of the Information System

The context of the system is represented by a context diagram that shows the interactions between the domain of study and its exchanges with the environment (Figure 2.1).

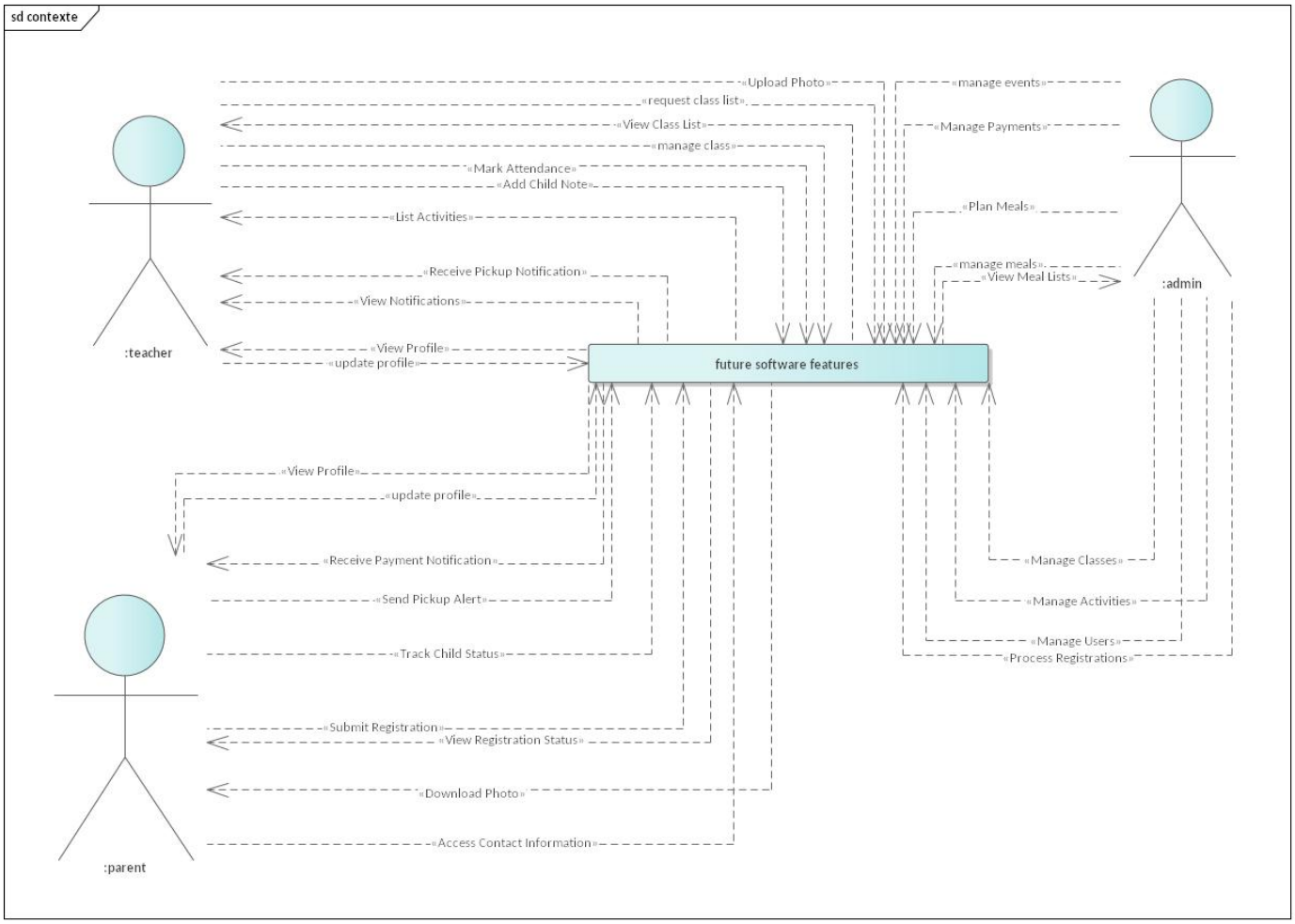



**Figure 2.1:** Context Diagram of the Information System

## 2.4 Identification of Use Cases

This section presents the global use case diagram of the Petit Suivi system, identifying all functional interactions between actors and the system.

### 2.4.1 Use Case Diagram

Figure 2.2 presents the global use case diagram, illustrating the relationships between the three actors (Parent, Teacher, and Administrator) and the system's functionalities. All use cases include the "Authenticate" use case as a precondition.

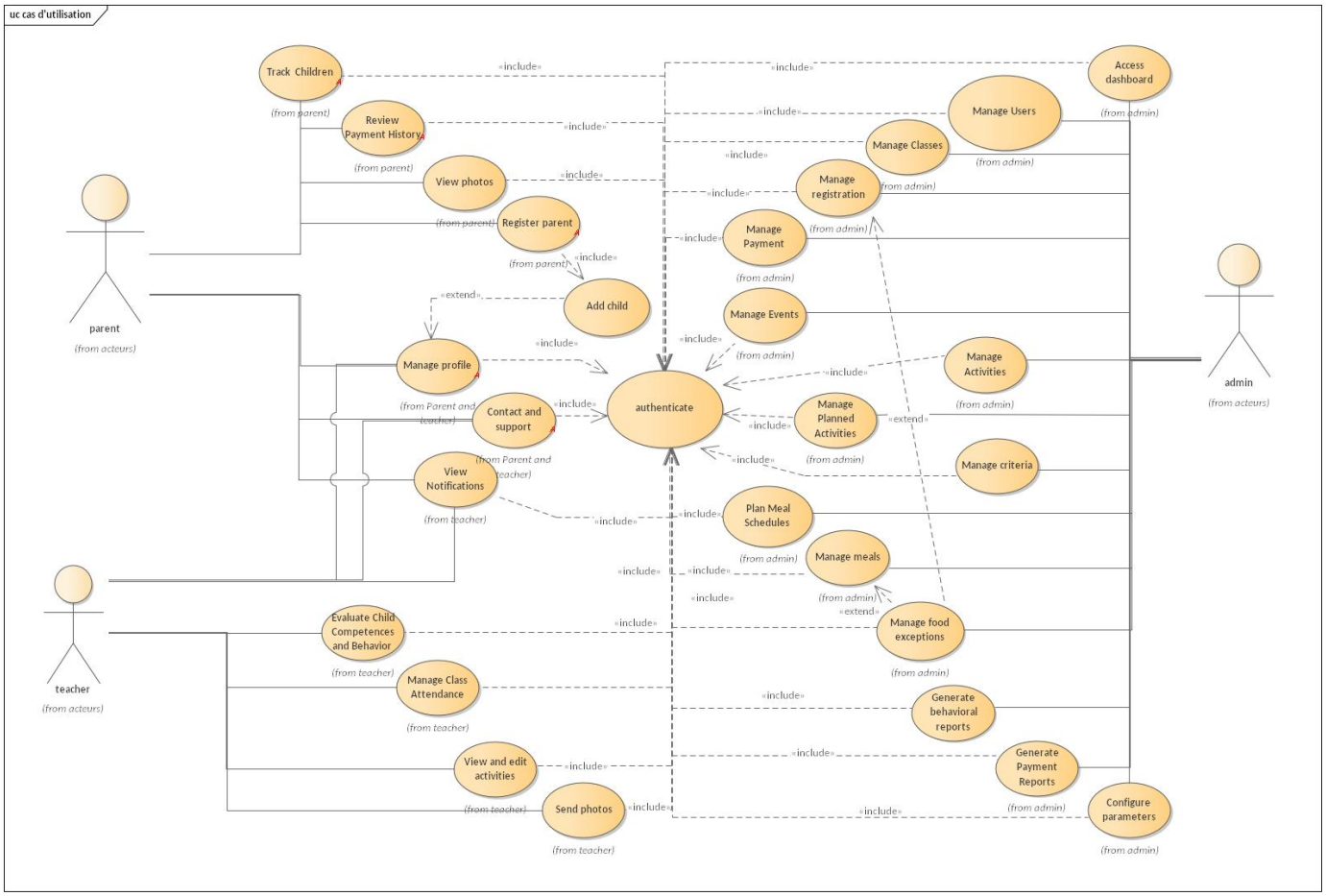



**Figure 2.2:** Global Use Case Diagram of the Petit Suivi System

## 2.5 Textual Description of Use Cases

This section provides a detailed textual description for each use case, following a structured format that specifies the actors involved, preconditions, the basic flow (step-by-step interaction between the actor and the system), and alternate or exception flows where applicable.

### 2.5.1 1. Parent Use Cases

**Use Case: 1.1 Track Children**

| | |
|---|---|
| **Actor** | Parent |
| **Objective** | This use case allows the parent to view today's scheduled activities, check daily attendance status, and review acquired competences for their child. |
| **Pre-condition** | The Parent is authenticated in the system.<br>The child's registration is approved. |
| **Post-condition** | The Parent has successfully reviewed the requested tracking information for the selected child. |
| **Nominal Scenario 1: Consult today's scheduled activities** | This scenario starts when the Parent accesses tracking section:<br>• The system displays the active enrollments of the Parent's children. If no active registration is found, then **Exception1**.<br>• The Parent selects a child.<br>• The system displays today's scheduled activities for the selected child. If no activities are scheduled for today, then **Exception2**. |
| **Nominal Scenario 2: Check daily attendance status** | This scenario starts when the Parent accesses tracking section:<br>• The system displays the active enrollments of the Parent's children. If no active registration is found, then **Exception1**.<br>• The Parent selects a child.<br>• The system displays the attendance records. |
| **Nominal Scenario 3: Review acquired competences** | This scenario starts when the Parent accesses tracking section:<br>• The system displays the active enrollments of the Parent's children. If no active registration is found, then **Exception1**.<br>• The Parent selects a child.<br>• The system displays the child's competences. If no competence found, then **Exception3**. |

| | |
|---|---|
| **Nominal Scenario 4: Send pickup notification** | This scenario starts when the Parent wants to notify the school of their arrival:<br>• The system displays the active enrollments of the Parent's children. If no active registration is found, then **Exception1**.<br>• The Parent selects a child.<br>• The Parent clicks on the "Send Pickup Notification" button for a specific child.<br>• The system prompts the Parent to enter the estimated duration until their arrival.<br>• The system sends a notification to the respective teachers of the child's class.<br>• The system displays a confirmation message: "Notification sent successfully." |
| **Exceptions** | **Exception1:** The system displays the message: "Please re-register for the new year."<br>**Exception2:** The system displays the message: "No activity scheduled for today."<br>**Exception3:** The system displays the message: "No competence found for this child." |

**Table 2.1:** Textual Description of Use Case 1.1 – Track Children

**Use Case: 1.2 Review Payment History**

| | |
|---|---|
| **Actor** | Parent |
| **Objective** | This use case allows the parent to review payment history. |
| **Pre-condition** | The Parent is authenticated. |
| **Post-condition** | The Parent has reviewed the payment history of the selected child. |
| **Nominal Scenario 1: Consult monthly payment records of a child** | This scenario starts when the Parent accesses the Payments section:<br>• The system displays the enrollments of the Parent's children. If no registration is found, then **Exception1**.<br>• The Parent selects a child.<br>• The system displays the payment history and monthly schedule overview. |
| **Exceptions** | **Exception1:** The system displays the message: "No registrations found." |

**Table 2.2:** Textual Description of Use Case 1.2 – Review Payment History

**Use Case: 1.3 View Photo**

| | |
|---|---|
| **Actor** | Parent |
| **Objective** | This use case allows the parent to view photos. |
| **Pre-condition** | The Parent is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Browse photo gallery of a child** | This scenario starts when the Parent accesses the Photo section:<br>• The system displays the active enrollments of the Parent's children. If no active registration is found, then **Exception2**.<br>• The Parent selects a child.<br>• The system displays the child's photos, providing a "Download" button for each photo. If no photos are available, then **Exception1**.<br>• The Parent taps the "Download" button on a specific photo.<br>• The system downloads the image. |
| **Exceptions** | **Exception1:** The system displays the message: "No photo available."<br>**Exception2:** The system displays the message: "Please re-register for the new year." |

**Table 2.3:** Textual Description of Use Case 1.3 – View Photo

### Use Case: 1.4 Register Parent

| | |
|---|---|
| **Actor** | Parent |
| **Objective** | This use case allows the parent to create a new account. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Submit registration form with parent and children information (new account)** | This scenario starts when the Parent wants to create a new account:<br>• The system displays the parent account creation form with fields for personal details and credentials.<br>• The Parent accesses the registration interface and enters their personal information (firstName, lastName, email, password, cin, phone, address, birthdate).<br>• The system validates the input fields:<br>– If any field is empty, then **Exception2**.<br>– If the CIN or email already exists, then **Exception1**.<br>– If the Parent is under 18 years old, then **Exception3**.<br>• The system proceeds with the child registration flow ⟨⟨include⟩⟩. **(See Use Case 1.5: Add Child Nominal Scenario 1: Register a new child with medical and registration information)**<br>• The system displays a success screen indicating the account is awaiting administration approval. |
| **Exceptions** | **Exception1:** The system displays an error message: "CIN or Email already exists."<br>**Exception2:** The system displays an error message: "All fields are mandatory."<br>**Exception3:** The system displays an error message: "Parent must be at least 18 years old." |

**Table 2.4:** Textual Description of Use Case 1.4 – Register Parent

### Use Case: 1.5 Add Child

| | |
|---|---|
| **Actor** | Parent |
| **Objective** | This use case allows the parent to add a child. |
| **Pre-condition** | The Parent is authenticated. |
| **Post-condition** | The operation is completed successfully. |

| | |
|---|---|
| **Nominal Scenario 1: Register a new child with medical and registration information** | This scenario starts when the Parent wants to add a child:<br>• The system displays the child information form with fields for basic child details, class type, and payment method.<br>• The Parent enters the basic information for their child (firstName, lastName, birthdate).<br>• The system validates the input:<br>– If any field is empty, then **Exception1**.<br>– If the child's age is not between 2 years and 5 years and 4 months, then **Exception2**.<br>• The system loads configuration parameters , including base fees, available class types, meal plans, and payment methods.<br>• The Parent selects the desired class type, payment method, and meal plan.<br>• The system displays the medical form section.<br>• The Parent fills in the medical information, and taps "Submit".<br>• The system creates the Child.<br>• The system creates a registration record with a "Pending" status.<br>• The system creates the submitted medical form.<br>• The system displays a success message. |
| **Alternate Scenario: Re-register an existing child** | This scenario starts when the Parent wants to re-register a child:<br>• The system displays the list of the Parent's children.<br>• The Parent selects a child.<br>• The Parent taps the "Re-register" button.<br>• The system displays the registration form.<br>• The Parent fills in the required information, selects the class type, payment method, and meal plan.<br>• The Parent clicks "Submit".<br>• The system creates a new registration with "Pending" status for the child.<br>• The system displays a success message. |
| **Exceptions** | **Exception1:** The system displays an error message: "All fields are mandatory."<br>**Exception2:** The system displays an error message: "Child must be between 2 years and 5 years 4 months old." |

**Table 2.5:** Textual Description of Use Case 1.5 – Add Child

### 2.5.2 2. Teacher Use Cases

**Use Case: 2.1 Evaluate Child Competences and Behavior**

| | |
|---|---|
| **Actor** | Teacher |
| **Objective** | This use case allows the teacher to evaluate child competences. |
| **Pre-condition** | The Teacher is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Assess each competence of an activity as acquired or needs reinforcement** | This scenario starts when the Teacher selects a child for evaluation:<br>• The system displays the child's profile.<br>• The system displays today's planned activities and their competence criteria. If no activities are scheduled for today, then **Exception1**.<br>• The Teacher marks competence criteria as "Acquired" or "Needs Reinforcement".<br>• The Teacher taps the "Submit Evaluations" button.<br>• The system saves the evaluation data and refreshes the view. |
| **Alternate Scenario: Submit a behavioral report for a child** | This scenario starts when the Teacher wants to add a behavioral report:<br>• The system displays the child's profile.<br>• The Teacher taps the "Add Behavioral Report" button.<br>• The system displays a behavioral report form.<br>• The Teacher writes an observation and selects a severity level.<br>• The Teacher taps the "Send Report" button.<br>• The system saves the behavioral report.<br>• The system sends notification to the parent. |
| **Exceptions** | **Exception1:** The system displays an error message: "No activities scheduled for today." |

**Table 2.6:** Textual Description of Use Case 2.1 – Evaluate Child Competences and Behavior

**Use Case: 2.2 Manage Class Attendance**

| | |
|---|---|
| **Actor** | Teacher |
| **Objective** | This use case allows the teacher to manage class attendance. |
| **Pre-condition** | The Teacher is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Record daily attendance for all children in a class** | This scenario starts when the Teacher navigates to the "Attendance" section:<br>• The system displays the list of enrolled children in the class. If the current day is Saturday or Sunday, then **Exception1**.<br>• The Teacher begins marking attendance by toggling each child's status switch to "Present" or "Absent".<br>• The Teacher taps the "Save Attendance" button.<br>• The system saves all attendance records and displays a success notification. |
| **Exceptions** | **Exception1:** The system displays an error message: "Attendance can only be recorded from Monday to Friday." |

**Table 2.7:** Textual Description of Use Case 2.2 – Manage Class Attendance

**Use Case: 2.3 View and Edit Activities**

| | |
|---|---|
| **Actor** | Teacher |
| **Objective** | This use case allows the teacher to view and edit activities. |
| **Pre-condition** | The Teacher is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Browse all planned activities of a class for the current academic year** | This scenario starts when the Teacher wants to view planned activities of a class:<br>• The system displays the assigned class(es) of the teacher.<br>• The Teacher selects a class.<br>• The system displays a calendar view of all planned activities for the current academic year.<br>• The Teacher selects a specific day.<br>• The system displays all planned activities scheduled for the selected day. If no activities are planned for the selected day, then **Exception1**.<br>• The Teacher taps on a specific activity to view its full details.<br>• The system displays the activity detail showing: title, description and an option to mark the activity status as "Executed" or "Not Executed". If the activity does not belong to today (past or future date), then **Exception2**.<br>• The Teacher marks the activity as "Executed" or "Not Executed". |

| | |
|---|---|
| **Exceptions** | **Exception1:** The system displays the message: "No activity planned for this day."<br>**Exception2:** The system displays the activity in read-only mode. |

**Table 2.8:** Textual Description of Use Case 2.3 – View and Edit Activities

### Use Case: 2.4 Send Photos

| | |
|---|---|
| **Actor** | Teacher |
| **Objective** | This use case allows the teacher to send photos. |
| **Pre-condition** | The Teacher is authenticated. |
| **Post-condition** | The selected photo is shared with the selected recipients. |
| **Nominal Scenario 1: Upload and share one photo for a specific child, an entire class, or a selected group of children** | This scenario starts when the Teacher navigates to the "Photos" section:<br>• The Teacher taps the "Add" button to initiate a new photo upload.<br>• The Teacher selects a photo from the gallery and the target audience (children). If no children are assigned to the teacher, then **Exception1**.<br>• The system uploads the image on the server.<br>• The system sends notifications to the relevant parents informing them that new photos are available for their child.<br>• The system displays a "Photos shared successfully" message. |
| **Exceptions** | **Exception1:** The system displays an error message: "No target recipients found." |

**Table 2.9:** Textual Description of Use Case 2.4 – Send Photos

### 2.5.3 3. Teacher and Parent Use Cases

**Use Case: 3.1 Contact and Support**

| | |
|---|---|
| **Actor** | Parent, Teacher |
| **Objective** | This use case allows the parent or teacher to consult contact information and support details. |
| **Pre-condition** | The Parent or Teacher is authenticated. |
| **Post-condition** | The requested contact channel is opened or the nursery contact information is displayed. |
| **Nominal Scenario 1: Consult nursery contact information** | This scenario starts when the Parent or Teacher wants to consult contact information:<br>• The system displays the Support page.<br>• The system displays contact details (Whatsapp, email, phone, etc.).<br>• The User taps on the email address displayed.<br>• The system opens the device's default email client. |

**Table 2.10:** Textual Description of Use Case 3.1 – Contact and Support

**Use Case: 3.2 Manage Profile**

| | |
|---|---|
| **Actor** | Parent, Teacher |
| **Objective** | This use case allows the parent or teacher to view and manage profile information. |
| **Pre-condition** | The Parent or Teacher is authenticated. |
| **Post-condition** | The profile information has been viewed or updated successfully. |
| **Nominal Scenario 1: View personal profile details (Parent)** | This scenario starts when the Parent taps on the "Profile" section:<br>• The system displays the Profile page with the Parent's personal information.<br>• The system displays the associated children linked to the Parent account.<br>• The Parent scrolls down to view additional linked sections such as change password or logout. |
| **Nominal Scenario 2: View personal profile details (Teacher)** | This scenario starts when the Teacher taps on the "Profile" section:<br>• The system displays the Profile page with the Teacher's personal information.<br>• The system displays the assigned class linked to the Teacher account.<br>• The Teacher scrolls down to view additional linked sections such as change password or logout. |
| **Alternate Scenario 1: Add a new child (Parent)** | This scenario starts when the Parent wants to add a new child from their profile:<br>• The Parent taps the "Add a Child" button on their profile page.<br>• The system proceeds with the child registration flow ⟨⟨extend⟩⟩. **(See Use Case 1.5: Add Child Nominal Scenario 1: Register a new child with medical and registration information)** |
| **Alternate Scenario 2: Re-register an existing child (Parent)** | This scenario starts when the Parent wants to re-register an existing child from their profile:<br>• The system displays the list of the Parent's children on the profile page.<br>• The Parent selects a child and taps the "Re-register" button.<br>• The system proceeds with the re-registration flow ⟨⟨extend⟩⟩. **(See Use Case 1.5: Add Child Alternate Scenario: Re-register an existing child)** |
| **Alternate Scenario 3: Update account password** | This scenario starts when the User (Teacher or Parent) taps the "Change Password" button:<br>• The system displays a password change form.<br>• The User enters and confirms the new password.<br>• The system validates the input. If it doesn't match or fails requirements, then **Exception1**.<br>• The system saves the new password and displays a success message. |
| **Exceptions** | **Exception1:** The system displays an error message. |

**Table 2.11:** Textual Description of Use Case 3.2 – Manage Profile

**Use Case: 3.3 View Notifications**

| | |
|---|---|
| **Actor** | Parent, Teacher |
| **Objective** | This use case allows the parent or teacher to view notifications. |
| **Pre-condition** | The Parent or Teacher is authenticated. |
| **Post-condition** | The selected notifications have been viewed and their read status has been updated if needed. |
| **Nominal Scenario 1: Browse and read received notifications** | This scenario starts when the User taps the Notification section:<br>• The system displays the list of notifications. If no notifications are found, then **Exception1**.<br>• The User scrolls through the list and selects an unread notification.<br>• The system marks the notification as read and displays its content. |
| **Exceptions** | **Exception1:** The system displays an error message: "No notifications found." |

**Table 2.12:** Textual Description of Use Case 3.3 – View Notifications

### 2.5.4 4. Admin Use Cases

**Use Case: 4.1 Access Dashboard**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to access dashboard statistics and indicators. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The Administrator has reviewed the dashboard data for the selected academic year. |
| **Nominal Scenario 1: View aggregated statistics across all academic years** | This scenario starts when the Administrator wants to view the dashboard:<br>• The system displays the main Dashboard page with summary statistics.<br>• The Administrator reviews key indicators and metrics. |

**Table 2.13:** Textual Description of Use Case 4.1 – Access Dashboard

**Use Case: 4.2 Manage Users (Parent, Teacher)**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to manage users (parent, teacher). |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Create a new teacher account** | This scenario starts when the Admin wants to create a new teacher account:<br>• The system displays the teacher creation form.<br>• The Admin fills in the personal information and sets a temporary password.<br>• The system validates the input. If CIN or email already exists, then **Exception1**. If the teacher is under 18 years old, then **Exception2**.<br>• The system creates the account and sends login credentials via email.<br>• The system refreshes the teachers table. |
| **Nominal Scenario 2: Modify parent or teacher information** | This scenario starts when the Admin wants to modify a user's information:<br>• The system displays the edit form with pre-filled details.<br>• The Admin updates the fields and clicks "Update".<br>• The system validates the input. If CIN or email already exists, then **Exception1**.<br>• The system saves the data and refreshes the list. |
| **Nominal Scenario 3: Approve a pending parent account** | This scenario starts when the Admin wants to approve a pending parent account:<br>• The system displays the pending registrations list.<br>• The admin selects a pending parent account.<br>• The Admin reviews the parent's profile.<br>• The Admin clicks "Approve".<br>• The system updates the account status to "Approved" and enables mobile access.<br>• The system refreshes the parent table. |
| **Nominal Scenario 4: View users** | This scenario starts when the Admin wants to view the list of parents or teachers:<br>• The Admin navigates to the "Parents" or "Teachers" section.<br>• The system displays the corresponding list of users. |

| | |
|---|---|
| **Alternate Scenario: Archive or unarchive a teacher account** | This scenario starts when the Admin wants to archive or unarchive a teacher account:<br>• The system displays a list of teachers.<br>• The Admin selects a teacher to archive or unarchive.<br>• The Admin clicks the archive or unarchive button.<br>• if the teacher is archived, the system displays a confirmation dialog warning that archiving prevents login.<br>• The system refreshes the list. |
| **Alternate Scenario: Reject a pending parent account** | This scenario starts when the Admin wants to reject a pending parent account:<br>• The system displays the registrations list.<br>• The Admin selects a pending parent account.<br>• The Admin clicks "Reject" for the pending account.<br>• The system updates the status to "Rejected" and prevents login access.<br>• The system refreshes the table. |
| **Alternate Scenario: Archive or unarchive a parent account** | This scenario starts when the Admin wants to archive or unarchive a parent:<br>• The system displays the list of parents.<br>• The Admin selects a parent.<br>• The Admin selects the "Archive" or "Unarchive" option.<br>• The system toggles the archive status and prevents or restores login access.<br>• The system refreshes the list. |
| **Exceptions** | **Exception1:** The system displays an error message if the CIN or email already exists.<br>**Exception2:** The system displays an error message if the teacher is under 18 years old. |

**Table 2.14:** Textual Description of Use Case 4.2 – Manage Users (Parent, Teacher)

**Use Case: 4.3 Manage Classes**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to manage classes. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The class record is created, updated, archived, or unarchived successfully. |
| **Nominal Scenario 1: Create a new class** | This scenario starts when the Admin wants to create a new class:<br>• The system displays the class creation form.<br>• The Admin enters the name, type, capacity, and assigns a teacher.<br>• The system validates the input. If conflict exists, then **Exception1**.<br>• The system creates the class and displays a success notification. |
| **Nominal Scenario 2: Modify class details** | This scenario starts when the Admin wants to modify a class:<br>• The system displays the classes list.<br>• The Admin selects a class.<br>• The system displays the edit form with pre-filled details.<br>• The Admin updates the fields (name, teacher, capacity) and saves.<br>• The system validates the input. If conflict exists, then **Exception1**.<br>• The system updates the class record and refreshes the list. |
| **Nominal Scenario 3: View classes** | This scenario starts when the Admin wants to view the list of classes:<br>• The Admin selects the class type.<br>• The system displays the current academic year's classes for the selected type. |
| **Alternate Scenario: Archive or unarchive an existing class** | This scenario starts when the Admin wants to archive or unarchive a class:<br>• The system displays the classes list.<br>• The Admin selects a class.<br>• The Admin selects the "Archive" or "Unarchive" option.<br>• The system toggles the archive status and refreshes the display. |
| **Exceptions** | **Exception1:** The system displays an error if the class name already exists. |

**Table 2.15:** Textual Description of Use Case 4.3 – Manage Classes

**Use Case: 4.4 Manage Registrations**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to manage registrations. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Approve a pending registration request** | This scenario starts when the Admin wants to approve a pending registration request:<br>• The system displays the pending registrations list.<br>• The Admin selects a child and reviews the medical form.<br>• The Admin clicks "Approve".<br>• The system displays the class selection form.<br>• The Admin assigns the child to a class. If no capacity, then **Exception2**.<br>• The system triggers AI medical analysis. If the AI fails or is unavailable, then **Exception1**. Otherwise, it extracts health notes.<br>• The Admin reviews the AI output.<br>• The Admin modify dietary and health comments if needed.<br>• The Admin saves the dietary and health comments.<br>• The system approves the registration and assigns the class and updates status to approved. |
| **Nominal Scenario 2: View list of registrations** | This scenario starts when the Admin wants to view the list of registrations:<br>• The system displays the current academic year's registrations.<br>• The Admin can filter the list by status (Pending, Approved, Rejected). |
| **Alternate Scenario: Reject a pending registration request** | This scenario starts when the Admin wants to reject a pending registration request:<br>• The system displays the registrations list.<br>• The Admin selects a child.<br>• The Admin selects "Reject".<br>• The system updates the status to "Rejected" and refreshes the list. |
| **Alternate Scenario: Archive or unarchive a registration** | This scenario starts when the Admin wants to archive or unarchive a registration:<br>• The system displays the registrations list.<br>• The Admin selects a child.<br>• The Admin selects the "Archive" or "Unarchive" option.<br>• The system updates the registration status and refreshes the display. |
| **Exceptions** | **Exception1:** The system displays an alert: "Please disable the AI option to continue."<br>**Exception2:** The system blocks approval. |

**Table 2.16:** Textual Description of Use Case 4.4 – Manage Registrations

**Use Case: 4.5 Manage Payments**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to manage payments. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Record a monthly payment as received** | This scenario starts when the Admin selects a child's payment details:<br>• The system displays the detailed payment and monthly schedule (if monthly) or annual status.<br>• The Admin selects the target month or the annual option and confirms the payment.<br>• The system saves the transaction and updates the payment status and balance. |
| **Nominal Scenario 2: Mark registration fees as paid** | This scenario starts when the Admin identifies an unpaid registration fee:<br>• The Admin clicks the registration fee button in the payments list.<br>• The system marks the fee as paid and updates the payment record. |
| **Alternate Scenario 1: View and print payment invoice (Facture de paiment)** | This scenario starts when the Admin wants to generate a payment invoice:<br>• The system displays the payments list.<br>• The Admin selects a payment record.<br>• The Admin clicks the "View Invoice" button.<br>• The system generates and displays the payment invoice.<br>• The Admin clicks the "Print" button to print the document. |
| **Alternate Scenario 2: View and print annual registration fee receipt (Frais Annuel d'Inscription reçu)** | This scenario starts when the Admin wants to generate a registration fee receipt:<br>• The system displays the payments list.<br>• The Admin selects a payment record.<br>• The Admin clicks the "View Receipt" button.<br>• The system generates and displays the annual registration fee receipt.<br>• The Admin clicks the "Print" button to print the document. |
| **Alternate Scenario 3: View and print payment receipt (Reçu de Paiement)** | This scenario starts when the Admin wants to generate a payment receipt:<br>• The system displays the payments list.<br>• The Admin selects a payment record.<br>• The Admin clicks the "View Receipt" button.<br>• The system generates and displays the payment receipt.<br>• The Admin clicks the "Print" button to print the document. |

**Table 2.17:** Textual Description of Use Case 4.5 – Manage Payments

**Use Case: 4.6 Manage Events**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to manage events. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Create a new event** | This scenario starts when the Admin wants to create a new event:<br>• The system displays the event creation form.<br>• The Admin enters the event title, description, date, and time.<br>• The Admin selects the target children and teachers to notify.<br>• The system validates the input. If a scheduling conflict is detected, then **Exception1**.<br>• The system saves the event and sends notifications to the selected recipients.<br>• The system refreshes the events list. |
| **Nominal Scenario 2: Mark an event as executed** | This scenario starts when the Admin wants to update an event's status:<br>• The system displays the events list.<br>• The Admin selects an event still marked as "Pending".<br>• The Admin clicks "Mark as Executed".<br>• The system updates the status to "Executed" and refreshes the list. |
| **Nominal Scenario 3: View events list** | This scenario starts when the Admin wants to view the list of events:<br>• The system displays the list of events. |
| **Alternate Scenario: Delete an existing event** | This scenario starts when the Admin wants to delete an event:<br>• The system displays the list of all events.<br>• The Admin selects an event and clicks "Delete".<br>• The Admin confirms the deletion.<br>• The system removes the event and refreshes the list. |
| **Exceptions** | **Exception1:** The system displays a conflict alert. |

**Table 2.18:** Textual Description of Use Case 4.6 – Manage Events

**Use Case: 4.7 Manage Activities**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to manage activities. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Create a new activity** | This scenario starts when the Admin wants to create a new activity:<br>• The admin clicks on the "Add an Activity" button.<br>• The system displays the activity creation form.<br>• The Admin enters the activity title and description.<br>• The Admin optionally clicks "Auto-fill Criteria" to let the AI suggest relevant competences. If the AI fails or is unavailable, then **Exception4**.<br>• The system validates the input. If invalid, then **Exception1**.<br>• The Admin saves the activity. If a duplicate title exists, then **Exception2**.<br>• The system creates the record and refreshes the list. |
| **Nominal Scenario 2: View list of activities** | This scenario starts when the Admin wants to view the list of activities:<br>• The system displays the list of all defined activities. |
| **Alternate Scenario: Delete an existing activity** | This scenario starts when the Admin wants to delete an existing activity:<br>• The Admin navigates to the "Activities" section.<br>• The Admin selects an activity and clicks the "Delete" button.<br>• The system checks if the activity is scheduled. If so, then **Exception3**.<br>• The Admin confirms the deletion in the pop-up dialog.<br>• The system removes the activity and displays a success message. |
| **Alternate Scenario: Modify activity details** | This scenario starts when the Admin wants to modify an existing activity:<br>• The system displays the existing activities.<br>• The Admin clicks the "Edit" button.<br>• The system opens the edit form with the current data.<br>• The Admin updates the fields and clicks "Update".<br>• The Admin optionally clicks "Auto-fill Criteria" to let the AI suggest relevant competences. If the AI fails or is unavailable, then **Exception4**.<br>• The system validates the input. If invalid, then **Exception1**. If duplicate, then **Exception2**.<br>• The system updates the activity and refreshes the view. |

| | |
|---|---|
| **Exceptions** | **Exception1:** If the title is empty the system displays a validation error.<br>**Exception2:** If an activity with the same title already exists, the system displays an error message.<br>**Exception3:** The system blocks deletion and displays an error alert: "Deletion not possible"<br>**Exception4:** The system displays an alert: "Please disable the AI option to continue." |

**Table 2.19:** Textual Description of Use Case 4.7 – Manage Activities

**Use Case: 4.8 Manage Planned Activities**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to manage planned activities. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: View planned activities** | This scenario starts when the Admin wants to view the planned activities schedule:<br>• The system displays the planning calendar for Current School Year.<br>• The Admin selects a class.<br>• The Admin selects a day.<br>• The system shows the planned activities on that day. |
| **Nominal Scenario 2: Assign an activity to a day** | This scenario starts when the Admin wants to assign an activity to a planning day:<br>• The system displays the planning calendar.<br>• The Admin selects a class and a specific day.<br>• The system shows a form to select an activity for a specific time.<br>• The Admin selects an activity. If the selected activity collides with another one in time and date, the system prevents the assignment, then **Exception1**.<br>• The Admin saves the planning. If the school year is archived, then **Exception2**. |
| **Nominal Scenario 3: Remove an assigned activity from a day** | This scenario starts when the Admin wants to remove an activity from a planning day:<br>• The system displays the planning calendar.<br>• The Admin selects a class and a specific day.<br>• The system displays the list of activities for the selected day and class.<br>• The Admin selects the activity to remove.<br>• The system unassigns the activity and saves the planning. If the school year is archived, then **Exception2**. |
| **Exceptions** | **Exception1:** The system blocks the assignment and displays a temporal collision alert if an activity overlap is detected.<br>**Exception2:** If the school year is archived, the system blocks modification and displays an error alert: "Cannot modify archived year." |

**Table 2.20:** Textual Description of Use Case 4.8 – Manage Planned Activities

**Use Case: 4.9 Manage Criteria**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to manage criteria. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Add a new criterion** | This scenario starts when the Admin wants to add a new criterion:<br>• The Admin navigates to the "Criteria" section.<br>• The Admin clicks the "Add Criterion" button.<br>• The Admin enters the criterion name and description.<br>• The system validates the input. If duplicate, then **Exception1**.<br>• The Admin saves the criterion.<br>• The system creates the record and refreshes the list. |
| **Nominal Scenario 2: View list of criteria** | This scenario starts when the Admin wants to view the list of criteria:<br>• The system retrieves and displays the current list of criteria. |
| **Alternate Scenario: Delete an existing criterion** | This scenario starts when the Admin wants to delete an existing criterion:<br>• The Admin selects a criterion from the list.<br>• The Admin clicks the "Delete" button.<br>• The system checks if the criterion is linked. If so, then **Exception2**.<br>• The Admin confirms the deletion in the pop-up dialog.<br>• The system removes the criterion and displays a success message. |
| **Exceptions** | **Exception1:** The system blocks creation if a criterion with the same name already exists.<br>**Exception2:** The system blocks deletion if the criterion is linked to existing activities. |

**Table 2.21:** Textual Description of Use Case 4.9 – Manage Criteria

**Use Case: 4.10 Plan Meal Schedules**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to Plan Meal Schedules. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Compose a weekly meal plan** | This scenario starts when the Admin selects a target week in the Weekly Menu section:<br>• The system displays the calendar for the week.<br>• The Admin assigns a meal to each day.<br>• The system validates the catalog is not empty (if empty, then **Exception1**).<br>• The system validates the meal assignments against existing children's food exceptions.<br>• If conflicts are detected, the system displays the affected children and allows the Admin to modify the specific assignments.<br>• The Admin replaces the conflicting meals for each affected child with acceptable alternatives.<br>• The Admin saves the validated plan.<br>• The system saves the weekly meal plan assignments. |
| **Nominal Scenario 2: Save a meal plan as a reusable template** | This scenario starts when the Admin clicks the "Save as Template" button:<br>• The system prompts for a template name.<br>• The Admin enters a name and saves.<br>• The system validates the name is not a duplicate. If duplicate exists, then **Exception2**.<br>• The system saves the template. |
| **Nominal Scenario 3: Load a previously saved meal plan template** | This scenario starts when the Admin clicks the "Load a Template" button for a blank week:<br>• The system displays a list of saved templates.<br>• The Admin selects a template and confirms.<br>• The system auto-populates the calendar slots with the template data.<br>• The Admin saves the new schedule. |
| **Alternate Scenario: Delete a saved meal plan template** | This scenario starts when the Admin selects a template to delete:<br>• The system displays the list of saved templates.<br>• The Admin identifies a template and clicks the "Delete" icon.<br>• The Admin confirms the deletion in the dialog.<br>• The system removes the template and refreshes the template list. |
| **Exceptions** | **Exception1:** The system blocks meal assignment if the catalog is empty.<br>**Exception2:** The system displays an error if the template name already exists. |

**Table 2.22:** Textual Description of Use Case 4.10 – Plan Meal Schedules

**Use Case: 4.11 Manage Meals**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to manage meals. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Add a new meal to the catalog** | This scenario starts when the Admin wants to add a new meal to the catalog:<br>• The Admin selects the "Add a Meal" option and enters the meal name.<br>• The system validates the input. If duplicate exists, then **Exception1**.<br>• The system scans children's health comments to detect potential food-related issues for each child. If the AI fails or is unavailable, then **Exception2**.<br>• The system automatically saves any detected food exceptions for the affected children.<br>• The system creates the record and refreshes the catalog. |
| **Nominal Scenario 2: View meals** | This scenario starts when the Admin wants to view the list of meals:<br>• The system retrieves and displays the full catalog of meals. |
| **Alternate Scenario: Remove a meal** | This scenario starts when the Admin wants to remove a meal:<br>• The system displays the list of meals.<br>• The Admin selects a meal to remove.<br>• The system checks if the meal is assigned to any menu. If so, then **Exception1**.<br>• The Admin confirms the deletion in the pop-up dialog.<br>• The system removes the meal and associated food exceptions and refreshes the table. |
| **Exceptions** | **Exception1:** The system blocks the action and displays an error.<br>**Exception2:** The system displays an alert: "Please disable the AI option to continue." |

**Table 2.23:** Textual Description of Use Case 4.11 – Manage Meals

**Use Case: 4.12 Manage Food Exceptions**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to manage food exceptions. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Add a food exception for a child** | This scenario starts when the Admin wants to add a food exception:<br>• The system displays the list of children with their existing food exceptions.<br>• The Admin selects "Add an Exception" and selects the child and the forbidden meal.<br>• The system validates. If the exception already exists, then **Exception1**.<br>• The system saves the new food exception. |
| **Nominal Scenario 2: View food exceptions for a child** | This scenario starts when the Admin wants to view the list of food exceptions for a specific child:<br>• The system displays the list of children.<br>• The Admin selects a child from the list.<br>• The system retrieves and displays the current forbidden meals for that child. |
| **Alternate Scenario: Remove a food exception from a child** | This scenario starts when the Admin wants to remove a food exception:<br>• The system displays the list of children with their forbidden meals.<br>• The Admin selects a child.<br>• The system displays the list of food exceptions for the selected child.<br>• The Admin selects the food exception to remove.<br>• The system removes the food exception and refreshes the list. |
| **Exceptions** | **Exception1:** The system displays an error. |

**Table 2.24:** Textual Description of Use Case 4.12 – Manage Food Exceptions

**Use Case: 4.13 Generate Behavioral Reports**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to generate behavioral reports. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: Consult and analyze behavioral reports** | This scenario starts when the Admin navigates to the "Behavioral Reports" section:<br>• The system displays the list of children with their behavioral report counts and analysis status.<br>• The Admin clicks a child to view individual behavioral reports.<br>• The Admin clicks AI analysis button on unanalysed or all behavioral reports.<br>• The system processes and updates the analysis status. If the AI analysis fails, then **Exception1**. |
| **Nominal Scenario 2: View AI analysis history (Behavioral History)** | This scenario starts when the Admin clicks the "Behavioral History" section:<br>• The system displays summary statistics and the history of analyses. |
| **Exceptions** | **Exception1:** The system displays an error message: "AI analysis failed. Please disable it in parameter interface." |

**Table 2.25:** Textual Description of Use Case 4.13 – Generate Behavioral Reports

**Use Case: 4.14 Generate Payment Reports**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to generate payment reports. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition** | The operation is completed successfully. |
| **Nominal Scenario 1: View financial reports** | This scenario starts when the Admin navigates to the "Financial Reports" section:<br>• The system displays a bar chart of collected vs. expected payments per month.<br>• The system displays summary KPI cards (total collected, outstanding balance, collection rate).<br>• The system displays a detailed breakdown for each month of the school year, allowing the Admin to view individual payment statuses and identify which parents have paid for each specific month. |

**Table 2.26:** Textual Description of Use Case 4.14 – Generate Payment Reports

**Use Case: 4.15 Configure Params**

| | |
|---|---|
| **Actor** | Administrator |
| **Objective** | This use case allows the administrator to configure params. |
| **Pre-condition** | The Administrator is authenticated. |
| **Post-condition**<br><br>**Nominal Scenario 1: View configured system parameters** | The operation is completed successfully.<br><br>This scenario starts when the Admin opens the configuration page:<br>• The system displays the current values of all global parameters. |
| **Alternate Scenario 1: Update system configuration parameter values** | This scenario starts when the Admin accesses the configuration form:<br>• The system displays all editable global parameters.<br>• The Admin modifies parameter values and saves.<br>• The system updates the configuration and displays a success message. |
| **Alternate Scenario 2: Archive current academic year** | This scenario starts when the Admin clicks the "Archive Academic Year" button:<br>• The system displays a confirmation dialog warning of the irreversible action.<br>• The Admin confirms the operation.<br>• The system archives the academic year and all associated classes and registrations. |

**Table 2.27:** Textual Description of Use Case 4.15 – Configure Params

### 2.5.5 5. All Actors Use Cases

**Use Case: 5.1 Authenticate**

| | |
|---|---|
| **Actor** | All users |
| **Objective** | This use case allows all users to authenticate. |
| **Post-condition** | The user is authenticated and redirected to the appropriate interface according to their role. |
| **Nominal Scenario 1: Parent login via mobile application** | This scenario starts when the Parent opens the mobile application:<br>• The Parent enters their email, password, and selects the "Parent" role.<br>• The system validates the credentials. If incorrect, then **Exception1**.<br>• The system checks the account approval status. If pending, then **Exception2**. If rejected, then **Exception3**.<br>• The system checks if the account is archived. If archived, then **Exception3**.<br>• The system redirects the Parent to the Parent Dashboard. |

| | |
|---|---|
| **Nominal Scenario 2: Teacher login via mobile application** | This scenario starts when the Teacher opens the mobile application:<br>• The Teacher enters their email, password, and selects the "Teacher" role.<br>• The system validates the credentials. If incorrect or inactive, then **Exception1**.<br>• The system checks if the account is archived. If archived, then **Exception3**.<br>• The system redirects the Teacher to the Teacher Dashboard. |
| **Nominal Scenario 3: Admin login via web panel** | This scenario starts when the Admin accesses the web panel:<br>• The Admin enters their email and password and clicks "Sign In".<br>• The system validates the credentials. If invalid, then **Exception1**.<br>• The system displays the full Admin Dashboard. |
| **Nominal Scenario 4: Parent or Teacher forgot password via mobile application** | This scenario starts when the Parent or Teacher taps "Forgot Password?" on the login screen:<br>• The system prompts the user to enter their email address.<br>• The user enters their email and confirms. If not found, then **Exception1**.<br>• The system generates a temporary password and sends it by email. If send fails, then **Exception2**. |
| **Exceptions** | **Exception1:** The system displays an authentication error.<br>**Exception2:** The system displays a "Waiting for Approval" screen if the Parent account is not yet approved.<br>**Exception3:** The system blocks login and displays the message: "You cannot login, please contact administration." |

**Table 2.28:** Textual Description of Use Case 5.1 – Authenticate

# Conclusion

Throughout this chapter, we presented a functional modeling of our application using Use Case diagrams and detailed textual scenario descriptions. This analysis allowed us to break down the system into clearly defined functional areas organized around the three main actors: Parent, Teacher, and Administrator, and to identify the key interactions each actor has with the system.
The use case scenarios provided a comprehensive view of the system's expected behavior. In the next chapter, we will deepen this analysis by examining the system's architecture and data model, through sequence diagrams and the domain class diagram and physical data schema.

# Chapter 3

# Analysis and Design

## 3.1 Introduction

At this stage of our study, we cross the boundary that separates the functional world from the object world, formally beginning an analysis that allows us to identify the system's objects.
Analysis is a very important step during the implementation of a computerized system. It mainly consists of elaborating a domain class model that allows a smooth transition to true object modeling and illustrates both the structural and dynamic dimensions of the system by formalizing the scenarios of its use cases through participant class diagrams and sequence diagrams.

## 3.2 Realization of Use Cases

Use case realization is achieved through a combination of participant class diagrams and sequence diagrams. Participant class diagrams illustrate the structural subset of the system involved in a specific scenario, while sequence diagrams present the cooperation between these objects from a temporal point of view. Together, these diagrams provide a comprehensive model of the dynamic aspects of our complex use case scenarios across the connected platforms.

### 3.2.1 Parent: Track Children - Scheduled Activities

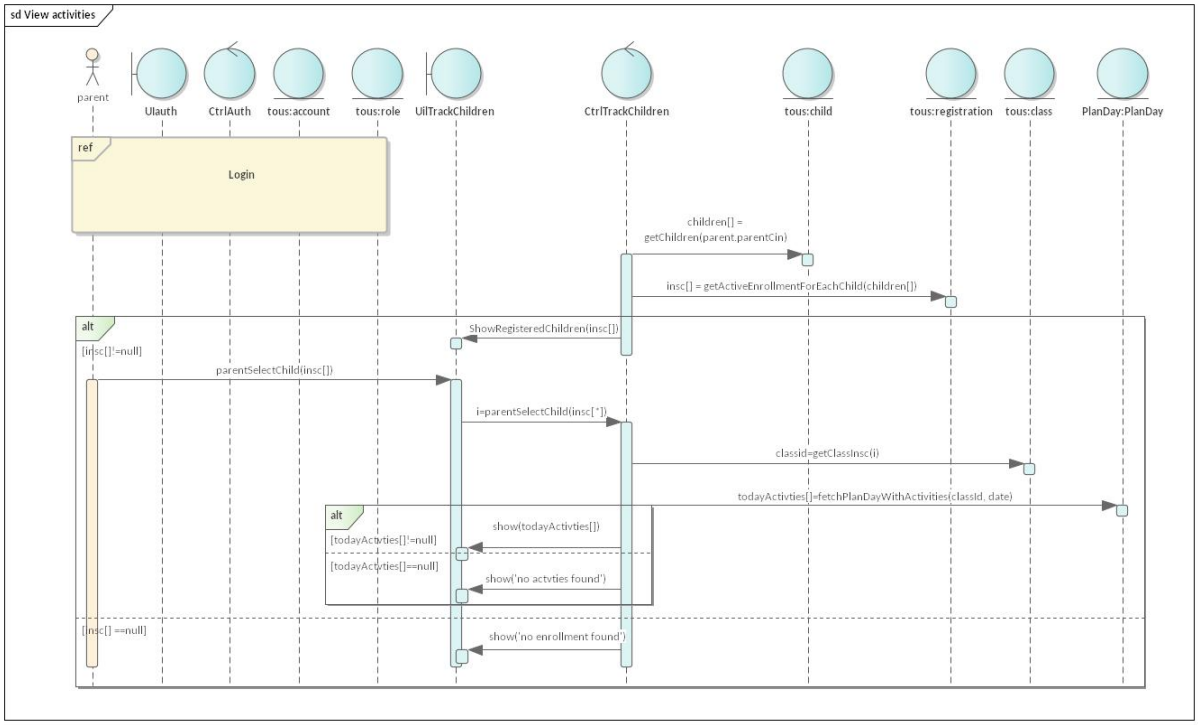



**Figure 3.1:** Sequence Diagram: Consult today's scheduled activities

### 3.2.2 Parent: Track Children - Attendance Status

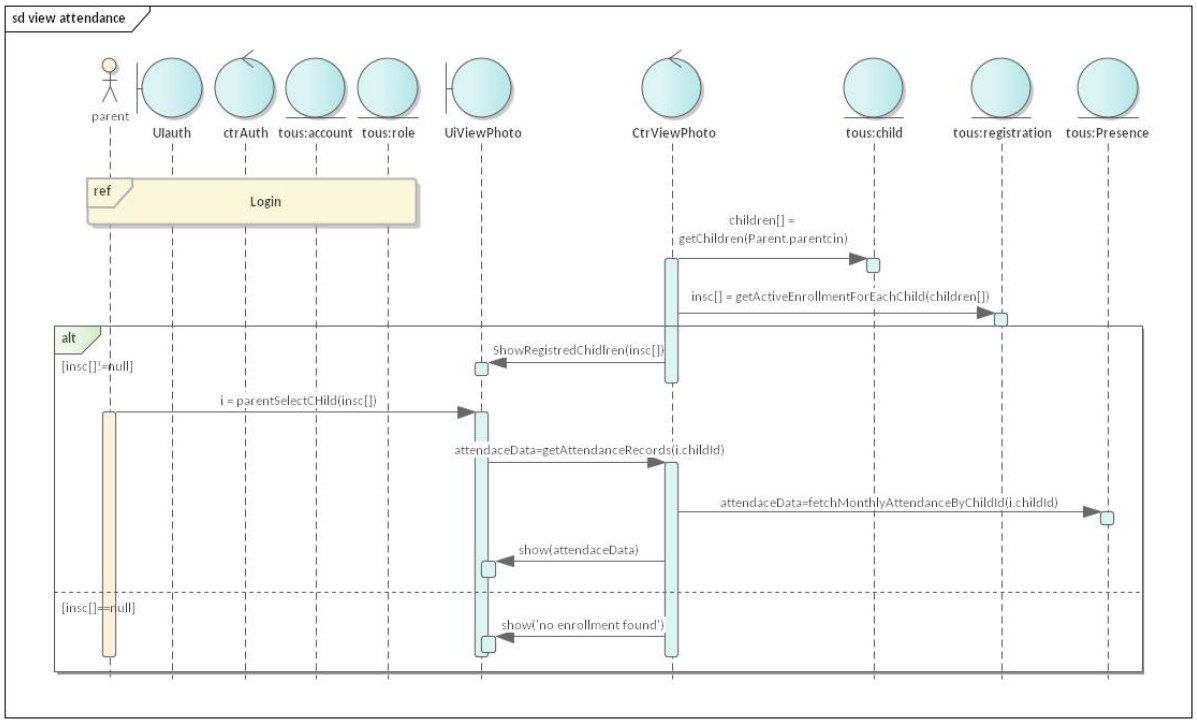



**Figure 3.2:** Sequence Diagram: Check daily attendance status

### 3.2.3 Parent: Track Children - Acquired Competences

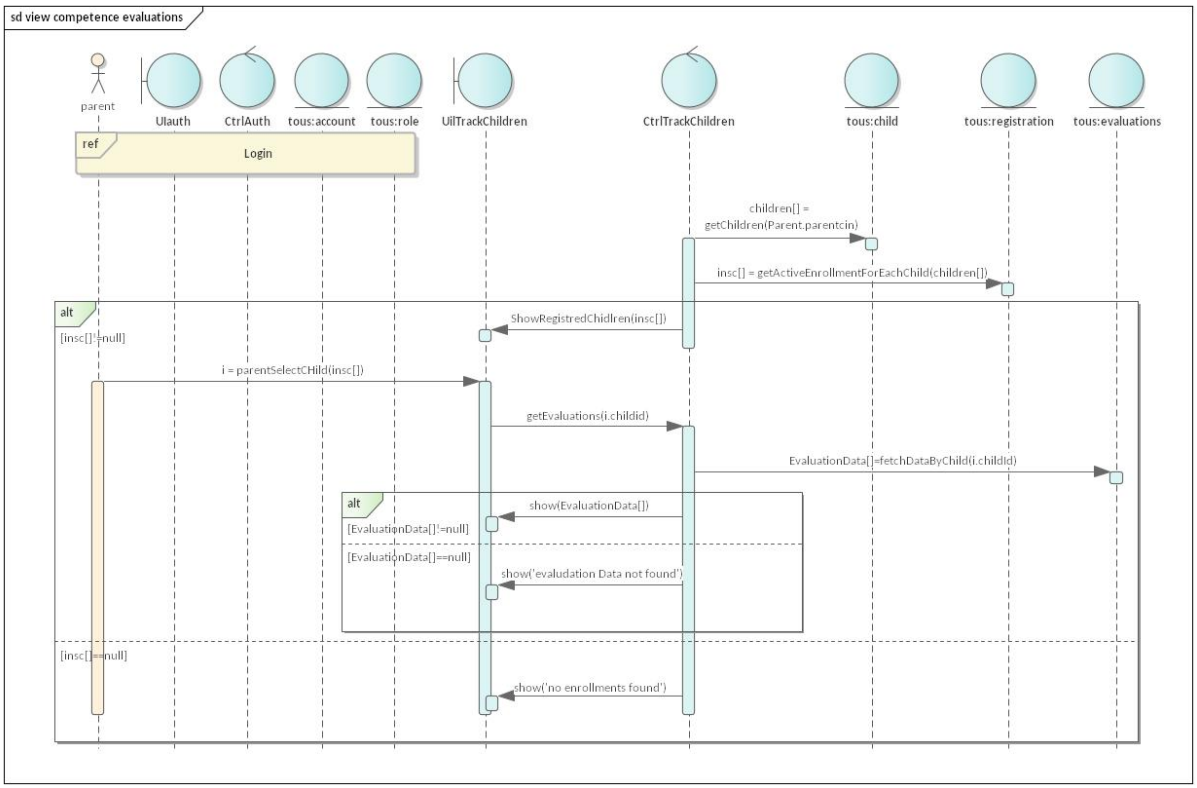



**Figure 3.3:** Sequence Diagram: Review acquired competences

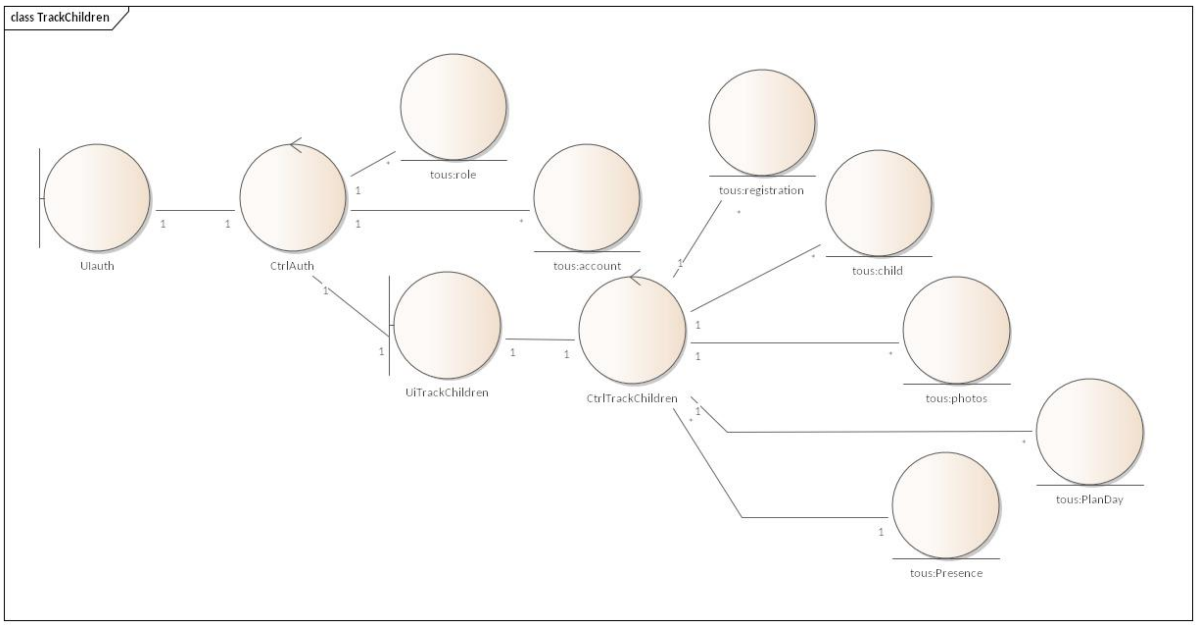



**Figure 3.4:** Class Diagram from UC: Track Children

### 3.2.4 Parent: View Photos

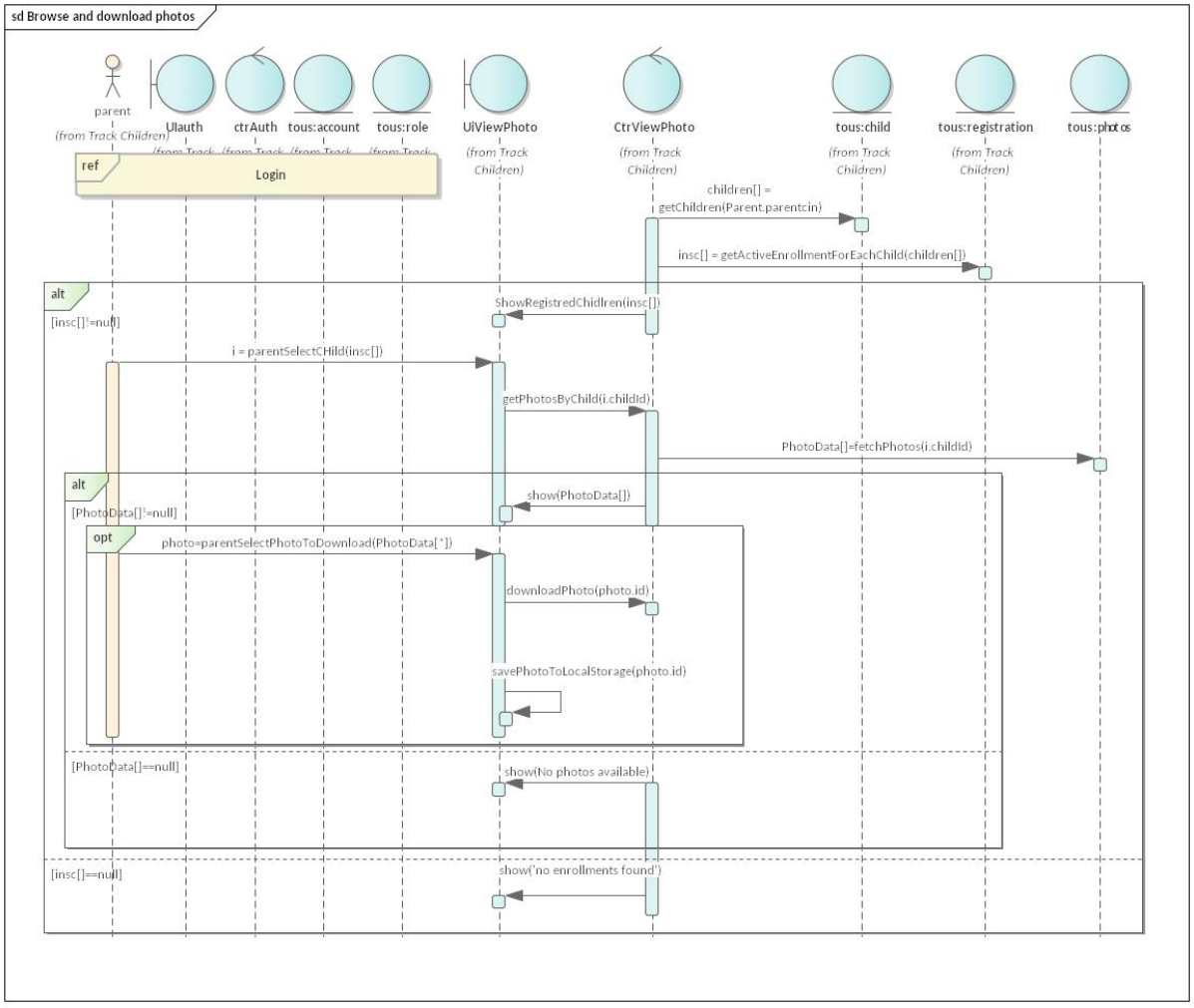



**Figure 3.5:** Sequence Diagram: Browse photo gallery

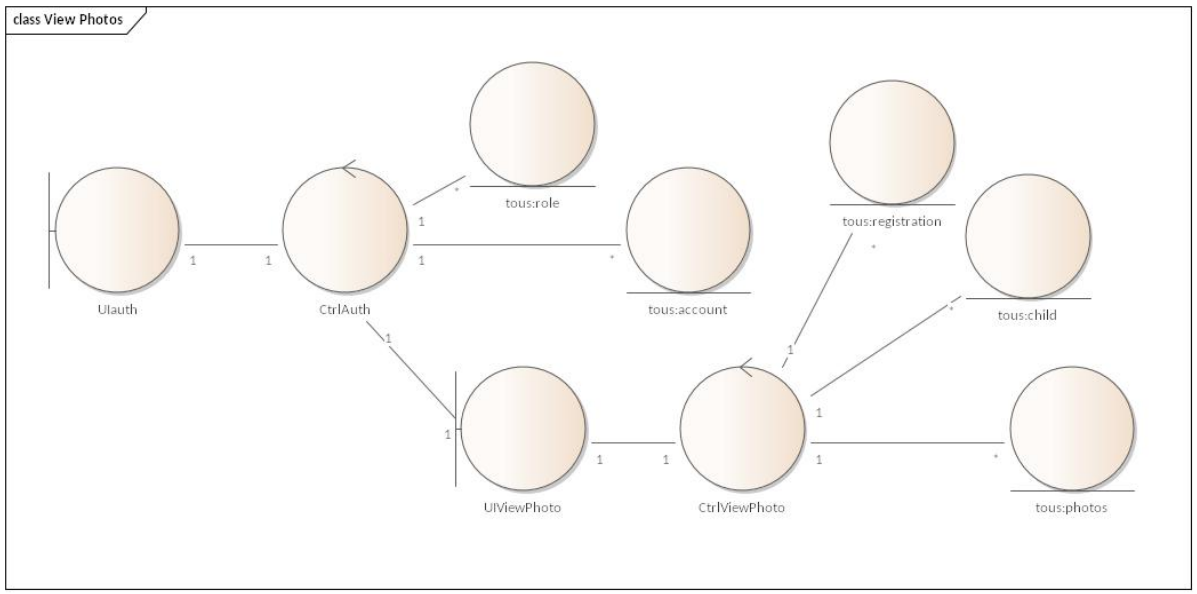



**Figure 3.6:** Class Diagram from UC: View Photos

### 3.2.5 Parent: Add Child - Register a new child with medical and registration information

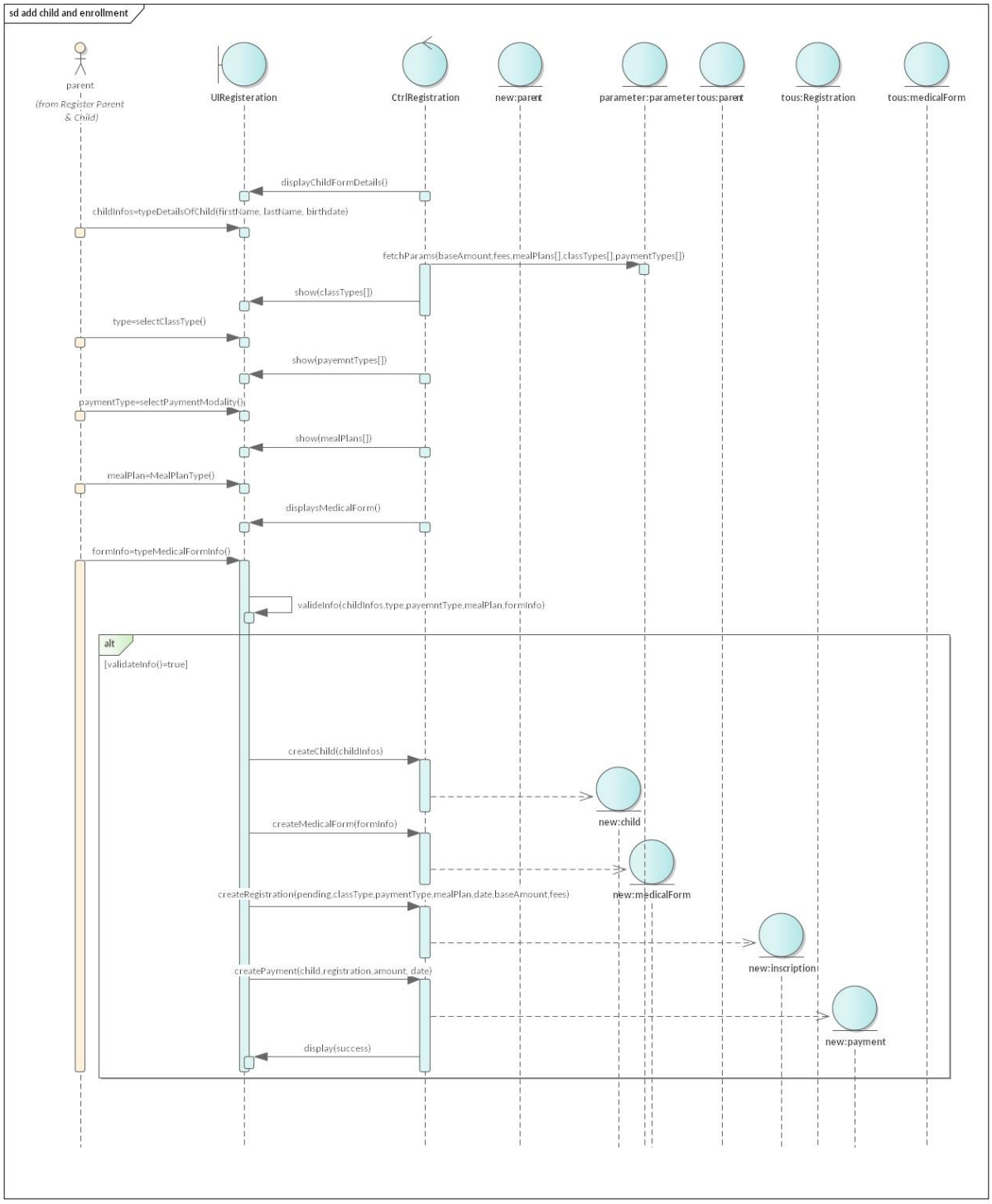



**Figure 3.7:** Class Diagram: Add Child and Enrollment

### 3.2.6 Parent: Register Parent - Submit Registration Form

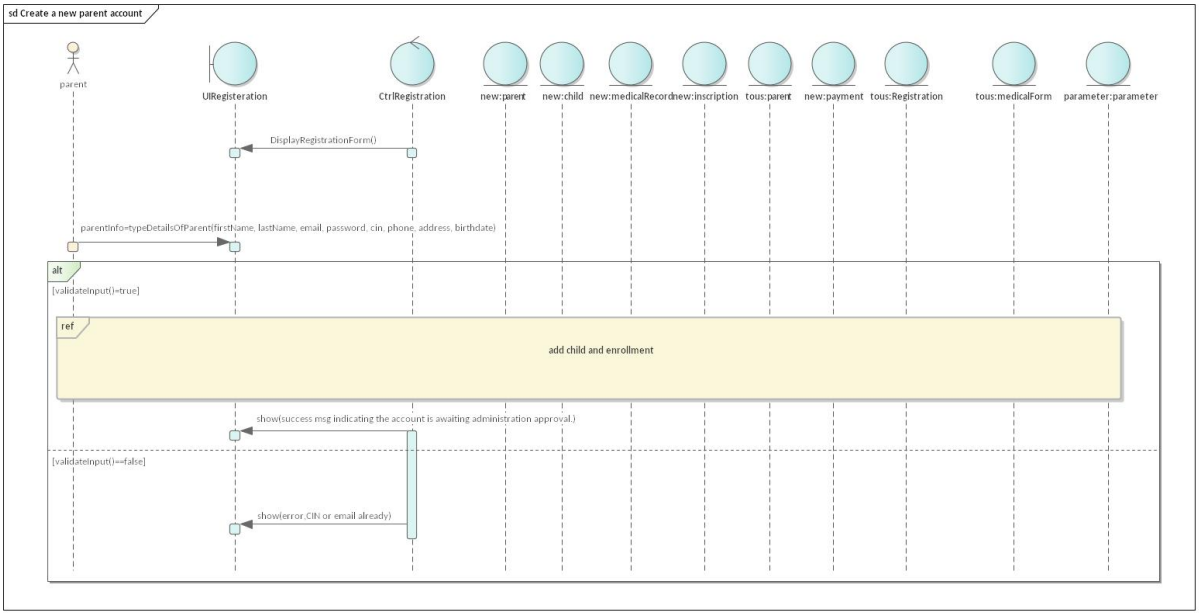



**Figure 3.8:** Class Diagram: Create a New Parent Account

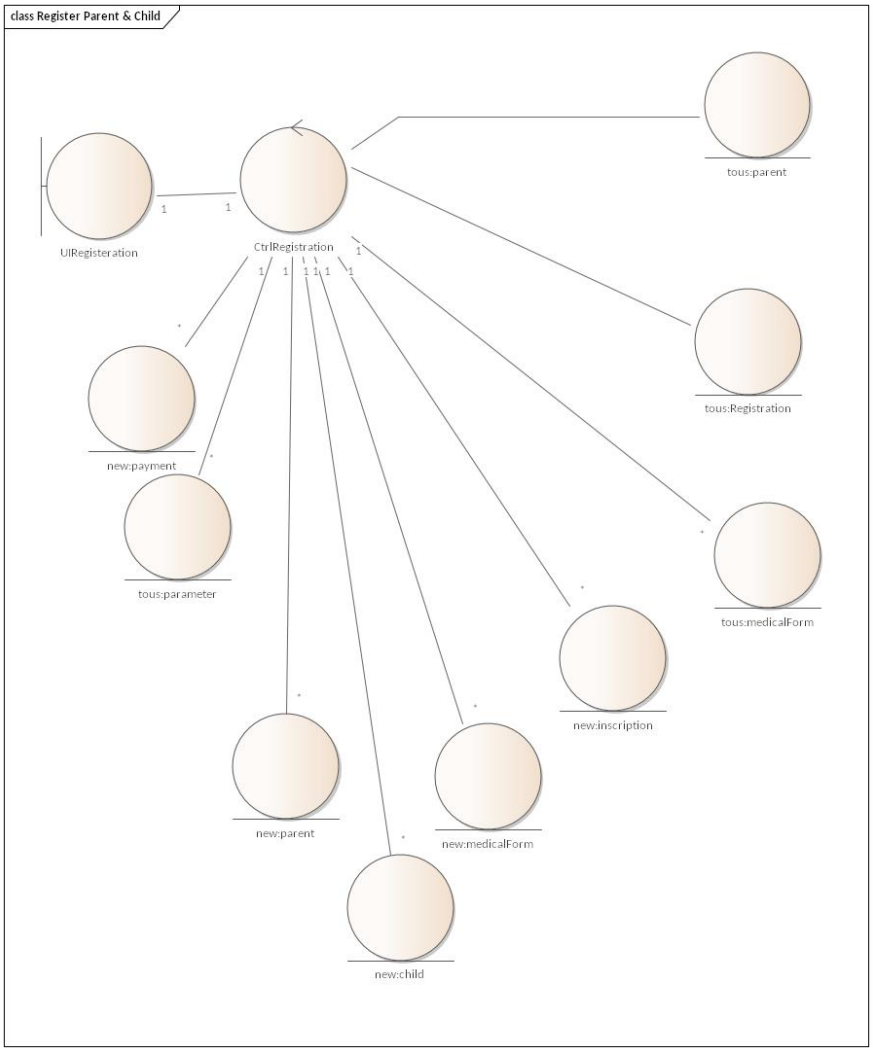



**Figure 3.9:** Class Diagram: Register Parent

### 3.2.7 Admin: Manage Registrations - Approve a Pending Registration Request

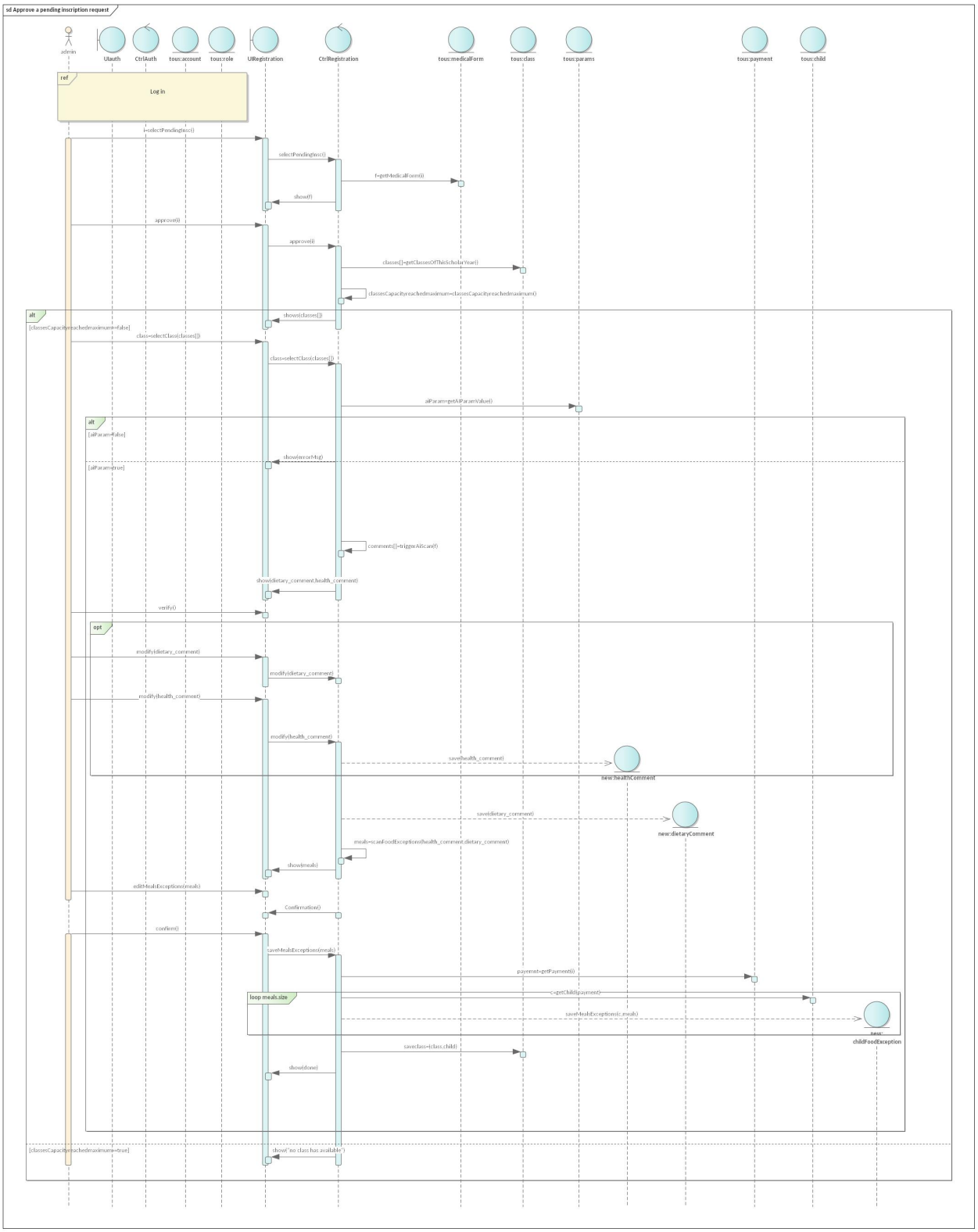



**Figure 3.10:** Sequence Diagram: Approve a pending registration request

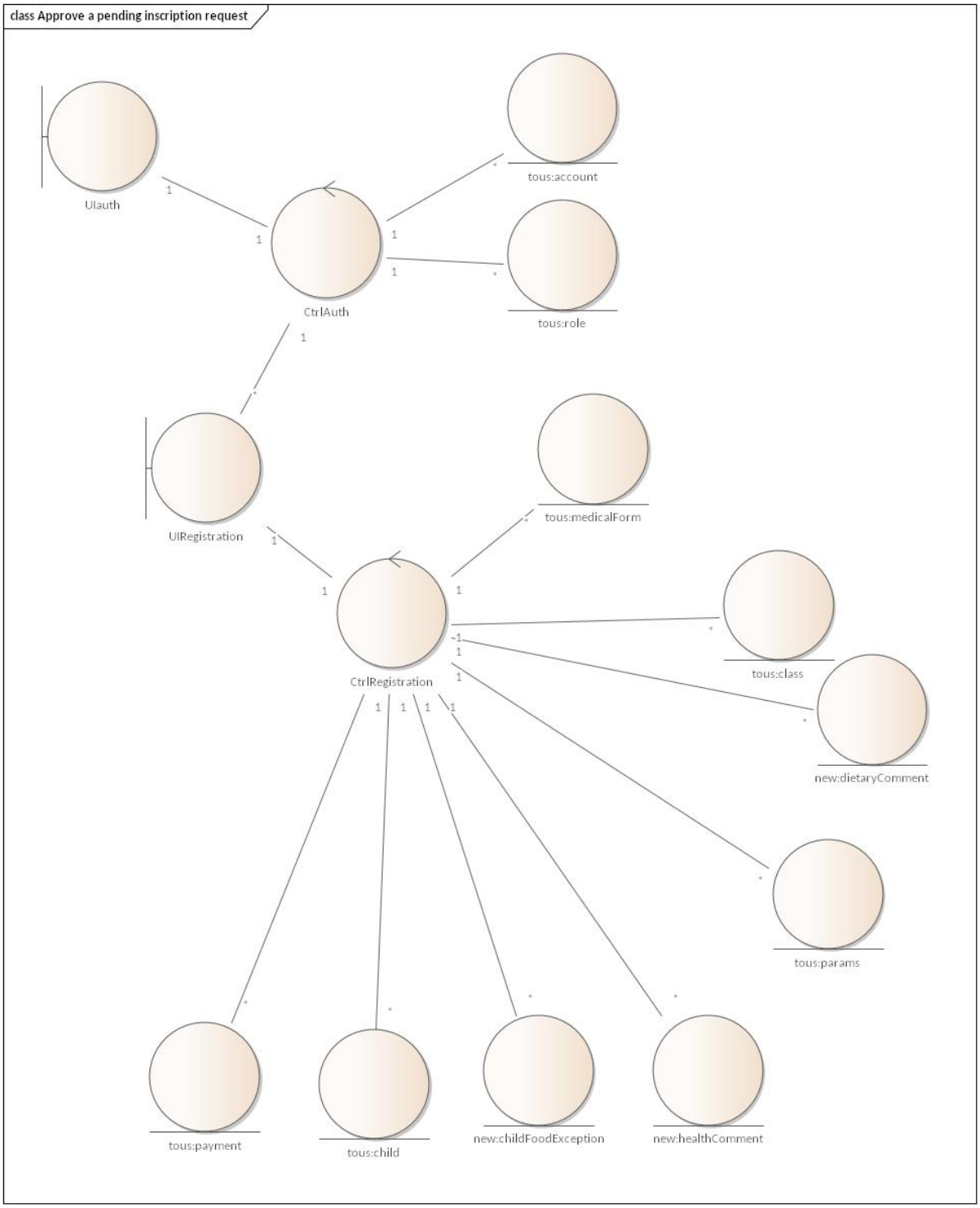



**Figure 3.11:** Class Diagram from UC: Manage Registrations

## 3.3 Domain Model Construction (Class Diagram)

The class diagram visualizes the static structure of the Petit Suivi system, displaying the primary object classes, their attributes, and their relational associations. This model forms the concrete blueprint upon which our database migrations and entity relationships are ultimately built.

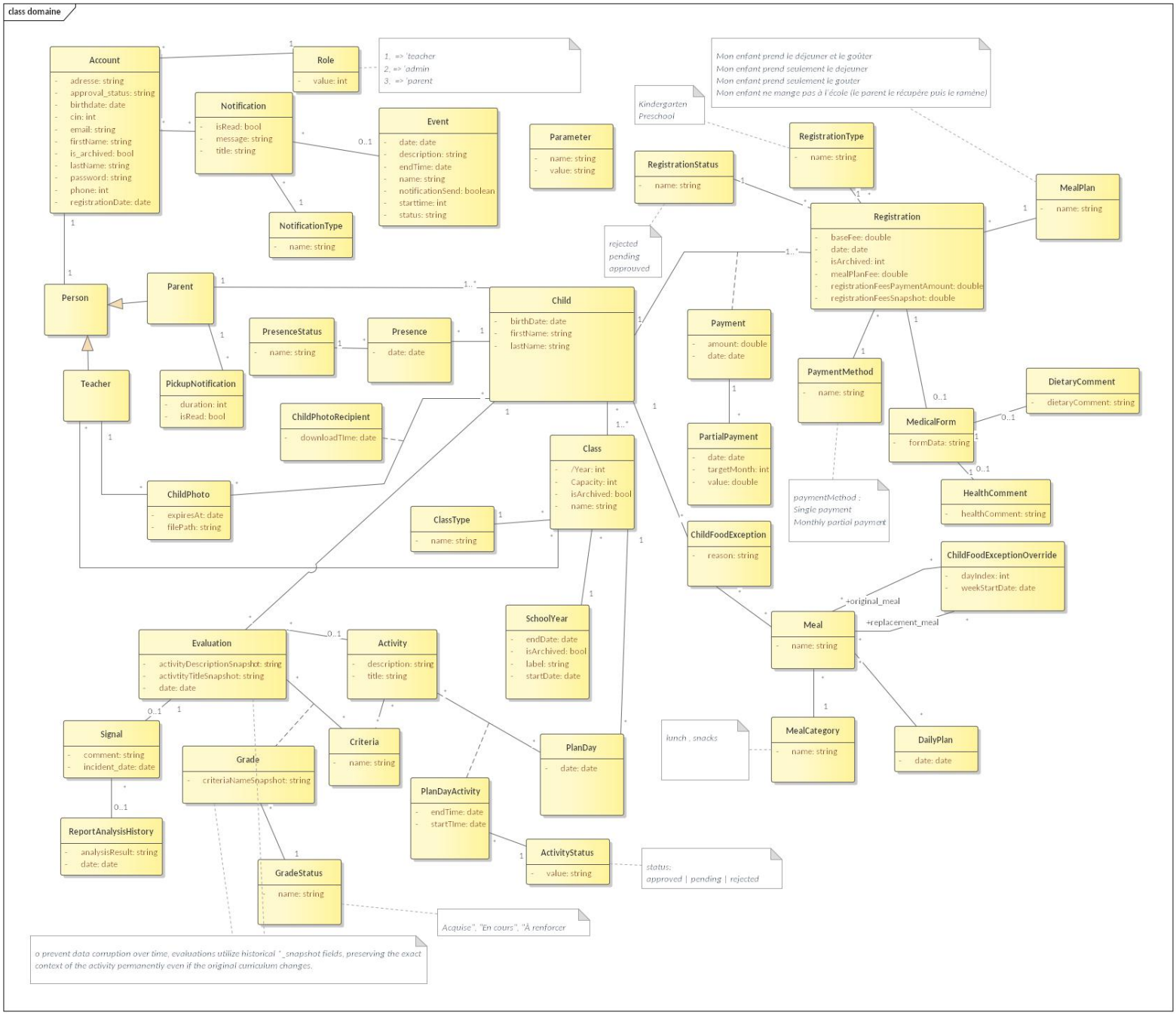



**Figure 3.12:** Global Class Diagram for Petit Suivi

### 3.3.1 Detailed Class Definitions

| # | Attribute Name | Description |
|---|---|---|
| 1 | **value** | Numeric value used to identify the role assigned to an account (1: teacher, 2: admin, 3: parent). |

**Table 3.1:** Class definition: Role

| # | Attribute Name | Description |
|---|---|---|
| 1 | **address** | Stores the account holder's postal address. |
| 2 | **approval_status** | Indicates whether the account has been approved. |
| 3 | **birthdate** | Stores the account holder's date of birth. |
| 4 | **cin** | Stores the national identity number linked to the account. |
| 5 | **email** | Stores the account email address. |
| 6 | **firstName** | Stores the account holder's first name. |
| 7 | **registrationDate** | Records the date when the account was created or registered. |
| 8 | **is_archived** | Indicates whether the account is archived. |
| 9 | **lastName** | Stores the account holder's last name. |
| 10 | **password** | Stores the encrypted password associated with the account. |
| 11 | **phone** | Stores the phone number associated with the account. |

**Table 3.2:** Class definition: Account

| # | Attribute Name | Description |
|---|---|---|
| 1 | **isRead** | Indicates whether the notification has been read. |
| 2 | **message** | Stores the main notification message shown to the user. |
| 3 | **title** | Stores the notification title. |

**Table 3.3:** Class definition: Notification

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the notification type name (e.g., alert, message). |

**Table 3.4:** Class definition: NotificationType

| # | Attribute Name | Description |
|---|---|---|
| 1 | **date** | Stores the date of the event. |
| 2 | **description** | Stores the detailed description of the event. |
| 3 | **endTime** | Stores the event ending time. |
| 4 | **name** | Stores the event title or name. |
| 5 | **notificationSend** | Indicates whether a notification should be sent for the event. |
| 6 | **starttime** | Stores the event starting time. |
| 7 | **status** | Stores the current status of the event. |

**Table 3.5:** Class definition: Event

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the parameter name. |
| 2 | **value** | Stores the value associated with the parameter. |

**Table 3.6:** Class definition: Parameter

| # | Attribute Name | Description |
|---|---|---|
| 1 | **dietaryComment** | Stores the dietary comment . |

**Table 3.7:** Class definition: DietaryComment

| # | Attribute Name | Description |
|---|---|---|
| 1 | **formData** | Stores the medical form data associated with the registration. |

**Table 3.8:** Class definition: MedicalForm

| # | Attribute Name | Description |
|---|---|---|
| 1 | **healthComment** | Stores the health-related . |

**Table 3.9:** Class definition: HealthComment

| # | Attribute Name | Description |
|---|---|---|
| 1 | **date** | Stores the date of the partial payment. |
| 2 | **targetMonth** | Identifies the month covered by the partial payment. |
| 3 | **value** | Stores the amount paid in the partial payment. |

**Table 3.10:** Class definition: PartialPayment

| # | Attribute Name | Description |
|---|---|---|
| 1 | **amount** | Stores the total payment amount. |
| 2 | **date** | Stores the payment date. |

**Table 3.11:** Class definition: Payment

| # | Attribute Name | Description |
|---|---|---|
| 1 | **baseFee** | Stores the base fee of the registration. |
| 2 | **date** | Stores the registration date. |
| 3 | **registrationFeeSnapshot** | Stores the registration fee snapshot captured at registration time. |
| 4 | **registrationFeesPaymentAmount** | Stores the amount paid for registration fees. |
| 5 | **isArchived** | Indicates whether the registration is archived. |
| 6 | **mealPlanFee** | Stores the fee associated with the selected meal plan. |

**Table 3.12:** Class definition: Registration

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the name of the payment method (e.g., Single payment, Monthly partial payment). |

**Table 3.13:** Class definition: PaymentMethod

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the registration type name (e.g., Kindergarten, Preschool). |

**Table 3.14:** Class definition: RegistrationType

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the meal plan name (e.g., Déjeuner et goûter, Seulement déjeuner, etc.). |

**Table 3.15:** Class definition: MealPlan

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the registration status name (e.g., rejected, pending, approved). |

**Table 3.16:** Class definition: RegistrationStatus

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the presence status name. |

**Table 3.17:** Class definition: PresenceStatus

| # | Attribute Name | Description |
|---|---|---|
| 1 | **date** | Stores the date of the recorded presence. |

**Table 3.18:** Class definition: Presence

| # | Attribute Name | Description |
|---|---|---|
| 1 | **expiresAt** | Stores the expiration date of the shared child photo. |
| 2 | **filePath** | Stores the file path of the child photo. |

**Table 3.19:** Class definition: ChildPhoto

| # | Attribute Name | Description |
|---|---|---|
| 1 | **birthDate** | Stores the child's date of birth. |
| 2 | **firstName** | Stores the child's first name. |
| 3 | **lastName** | Stores the child's last name. |

**Table 3.20:** Class definition: Child

| # | Attribute Name | Description |
|---|---|---|
| 1 | **downloadTime** | Stores the date when the recipient downloaded the child photo. |

**Table 3.21:** Class definition: ChildPhotoRecipient

| # | Attribute Name | Description |
|---|---|---|
| 1 | **duration** | Stores the duration during which the pickup notification remains valid. |
| 2 | **isRead** | Indicates whether the pickup notification has been read. |

**Table 3.22:** Class definition: PickupNotification

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the pickup notification status value. |

**Table 3.23:** Class definition: PickupNotificationsStatus

| # | Attribute Name | Description |
|---|---|---|
| 1 | **Capacity** | Stores the maximum number of children allowed in the class. |
| 2 | **isArchived** | Indicates whether the class is archived. |
| 3 | **name** | Stores the class name. |
| 4 | **Year** | Stores the academic year associated with the class. |

**Table 3.24:** Class definition: Class

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the class type name. |

**Table 3.25:** Class definition: ClassType

| # | Attribute Name | Description |
|---|---|---|
| 1 | **reason** | Stores the reason for the child's food exception. |

**Table 3.26:** Class definition: ChildFoodException

| # | Attribute Name | Description |
|---|---|---|
| 1 | **date** | Stores the date associated with the daily plan. |

**Table 3.27:** Class definition: DailyPlan

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the menu planning name. |

**Table 3.28:** Class definition: MenuPlanning

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the meal name. |

**Table 3.29:** Class definition: Meal

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the meal category name. |

**Table 3.30:** Class definition: MealCategory

| # | Attribute Name | Description |
|---|---|---|
| 1 | **dayIndex** | Stores the day index concerned by the food exception override. |
| 2 | **weekStartDate** | Stores the start date of the week for the override. |

**Table 3.31:** Class definition: ChildFoodExceptionOverride

| # | Attribute Name | Description |
|---|---|---|
| 1 | **endDate** | Stores the ending date of the school year. |
| 2 | **isArchived** | Indicates whether the school year is archived. |
| 3 | **label** | Stores the label of the school year. |
| 4 | **startDate** | Stores the starting date of the school year. |

**Table 3.32:** Class definition: SchoolYear

| # | Attribute Name | Description |
|---|---|---|
| 1 | **date** | Stores the date associated with the plan day. |

**Table 3.33:** Class definition: PlanDay

| # | Attribute Name | Description |
|---|---|---|
| 1 | **endTime** | Stores the ending time of the planned activity. |
| 2 | **startTime** | Stores the starting time of the planned activity. |

**Table 3.34:** Class definition: PlanDayActivity

| # | Attribute Name | Description |
|---|---|---|
| 1 | **value** | Stores the value of the activity status (e.g., approved, pending, rejected). |

**Table 3.35:** Class definition: ActivityStatus

| # | Attribute Name | Description |
|---|---|---|
| 1 | **description** | Stores the activity description. |
| 2 | **title** | Stores the activity title. |

**Table 3.36:** Class definition: Activity

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the criteria name. |

**Table 3.37:** Class definition: Criteria

| # | Attribute Name | Description |
|---|---|---|
| 1 | **activityDescriptionSnapshot** | Stores the activity description snapshot saved in the evaluation. |
| 2 | **activityTitleSnapshot** | Stores the activity title snapshot saved in the evaluation. |
| 3 | **date** | Stores the evaluation date. |

**Table 3.38:** Class definition: Evaluation

| # | Attribute Name | Description |
|---|---|---|
| 1 | **criteriaNameSnapshot** | Stores the criteria name snapshot saved in the grade. |

**Table 3.39:** Class definition: Grade

| # | Attribute Name | Description |
|---|---|---|
| 1 | **name** | Stores the grade status name (e.g., Acquise, En cours, À renforcer). |

**Table 3.40:** Class definition: GradeStatus

| # | Attribute Name | Description |
|---|---|---|
| 1 | **comment** | Stores the comment associated with the incident report. |
| 2 | **incident_date** | Stores the date of the reported incident. |

**Table 3.41:** Class definition: Signal

| # | Attribute Name | Description |
|---|---|---|
| 1 | **analysisResult** | Stores the generated report analysis result. |
| 2 | **date** | Stores the date of the analysis record. |

**Table 3.42:** Class definition: ReportAnalysisHistory

## 3.4 Future Software Architecture

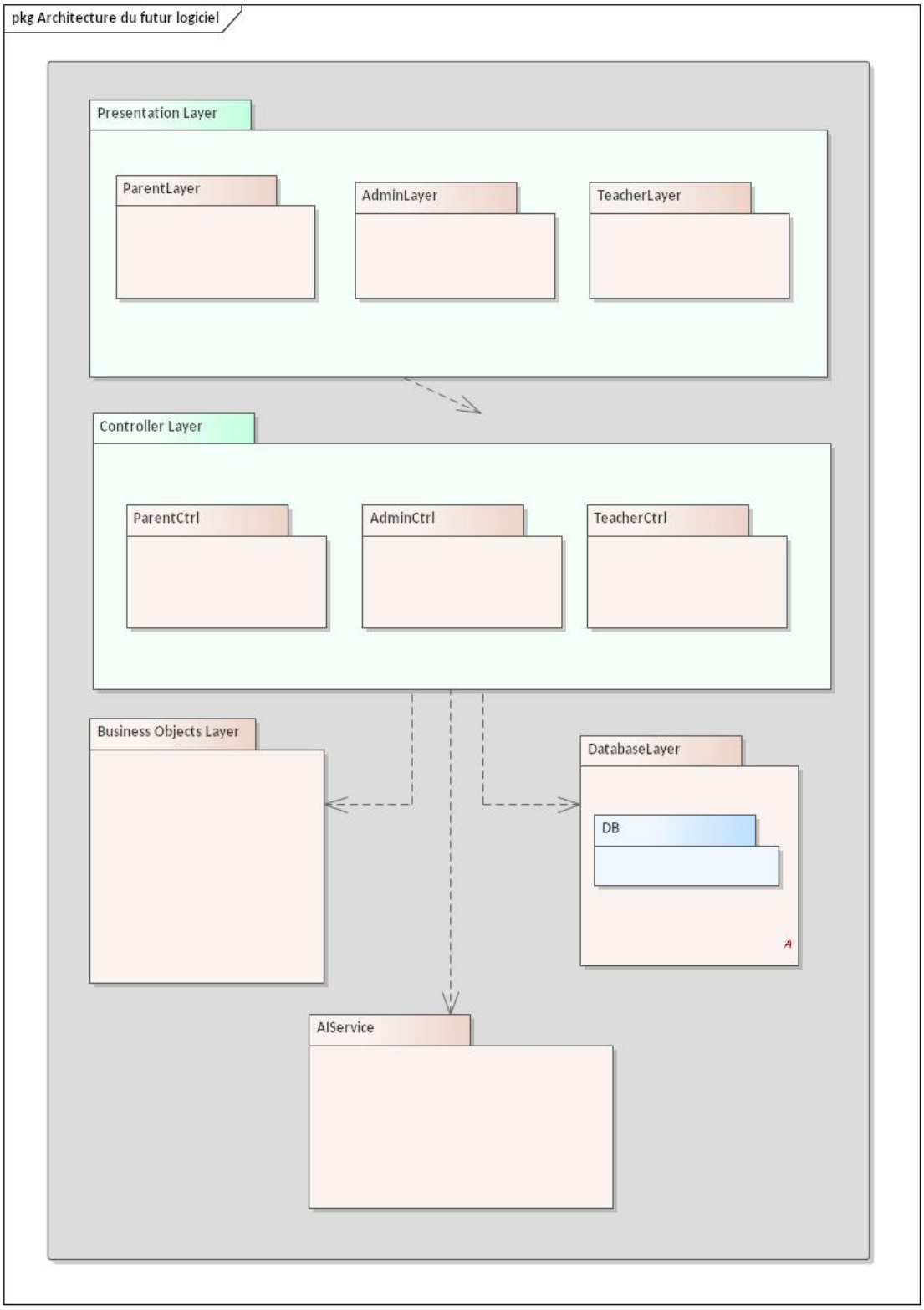



**Figure 3.13:** Future Software Architecture

## 3.5 Physical Data Schemas

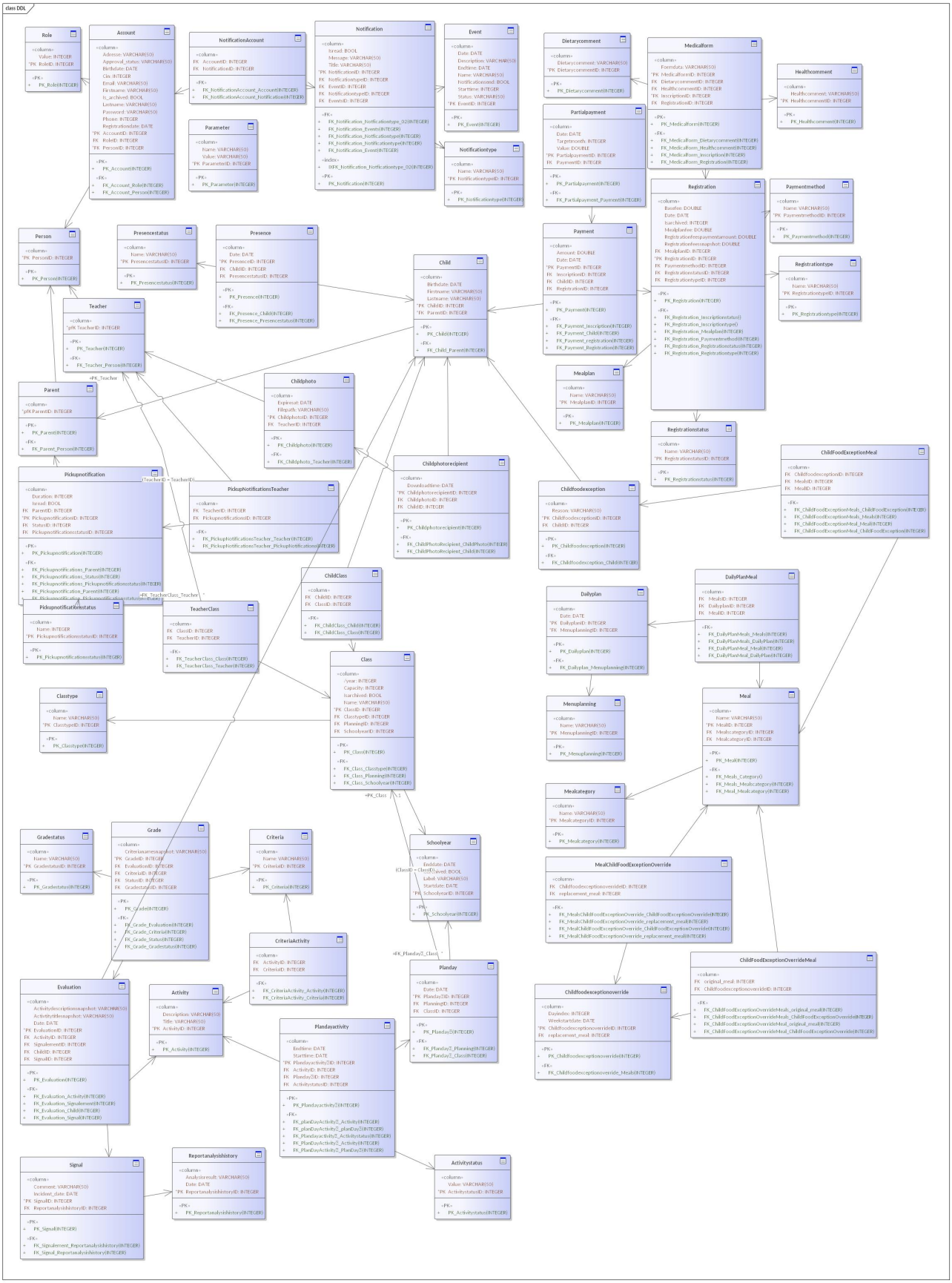



**Figure 3.14:** Physical Data Schemas

## 3.6 Conclusion

Through the formalization of the use cases with sequence diagrams and the structural definition provided by the class diagram, this chapter established a solid architectural foundation for the Petit Suivi system. This analysis ensures overall system coherence and provides a clear transition to the technical implementation phase presented in the next chapter.

# Chapter 4

# Implementation

## 4.1 Introduction

This chapter is dedicated to the realization of the **Petit Suivi** platform. It bridges the gap between our abstract modeling phase and the final, tangible product. We begin by defining the development environment, detailing the hardware specifications and the suite of software and technologies selected for developing the diverse components of the platform.

## 4.2 Development Environment

The development environment consists of the combination of hardware and software resources required to effectively build, test, and deploy the application.

### 4.2.1 Hardware Environment

- **Operating System:** Arch Linux (64-bit).
- **Processor:** High-performance multi-core processor.
- **Memory (RAM):** 16 GB.
- **Storage:** 512 GB SSD.

### 4.2.2 Software Environment and Technologies

#### Backend API

Laravel (PHP) and MySQL.

#### Frontend Web

React.js for the administrative dashboard.

#### Mobile Application

Flutter (Dart) for Parents and Teachers.

### AI Microservice

Node.js and GitHub Copilot SDK.

### Code Editor

vscode and Vim.

## 4.2.3 Deployment and Infrastructure Tools

### CI/CD Pipeline

GitHub CI/CD pipelines were configured to automate the deployment process. A self-hosted GitHub Actions runner, installed on the production server, listens for pushes to the `main` branch and automatically pulls the latest code and rebuilds the Docker containers.

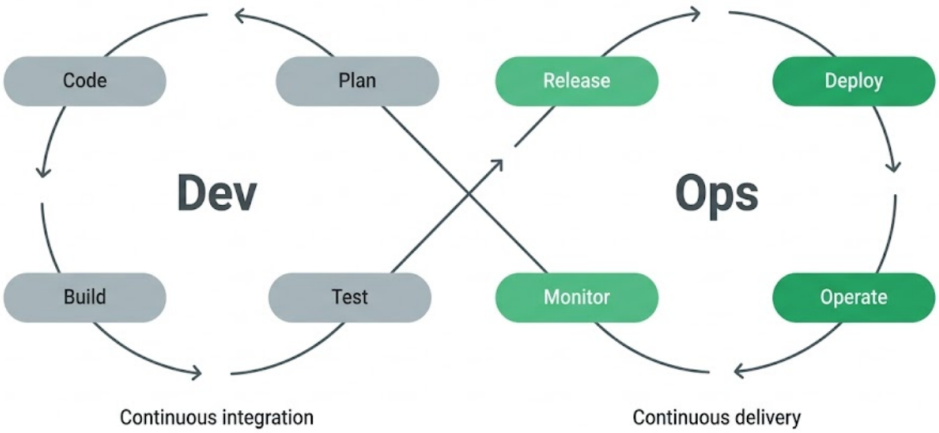



**Figure 4.1:** CI/CD Pipeline Configuration

### Domain and Web Accessibility

To make the platform publicly accessible, a custom domain name `petitsuivi.me` was purchased from **Namecheap**. The domain's DNS nameservers were then pointed to **Cloudflare**, which manages all DNS records and provides free SSL/TLS encryption.
To securely expose the locally hosted application to the internet without opening any firewall ports or revealing the server's public IP address, we used **Cloudflare Tunnel** (`cloudflared`). The tunnel runs as a Docker container alongside the application stack and establishes a persistent outbound connection from the local machine to the Cloudflare edge network. Cloudflare then routes incoming requests as follows:

- `petitsuivi.me` → React Frontend container.
- `api.petitsuivi.me` → Nginx/Laravel Backend container.

This architecture eliminates the need for traditional port forwarding or self-managed SSL certificates, while benefiting from Cloudflare's built-in DDoS protection and HTTPS enforcement.

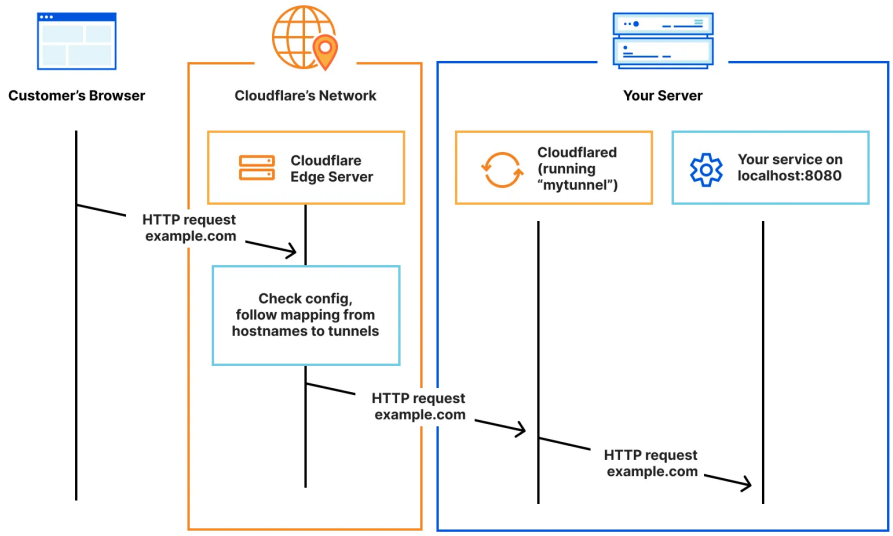



**Figure 4.2:** Cloudflare Tunnel Architecture

The key advantages of this architecture include:

- **Hidden Server IP:** The server's real IP address is never exposed to the public internet, making it invisible to attackers.
- **DDoS Protection:** All traffic passes through Cloudflare's edge network, which filters out malicious traffic and absorbs volumetric DDoS attacks before they reach the server.
- **No Open Ports:** Since the tunnel uses an outbound connection, no inbound firewall ports need to be opened, significantly reducing the attack surface.
- **Automatic HTTPS:** Cloudflare provides free SSL/TLS certificates, ensuring all communication between clients and the server is encrypted without any manual certificate management.
- **Anti-Spam and Bot Protection:** Cloudflare's built-in WAF (Web Application Firewall) helps block spam bots, brute-force attempts, and other automated threats.

## 4.3 Technical Implementation of Use Cases

### 4.3.1 Parent Use Cases

# Use Case name: Track Children

Interface for Viewing The track children dashboard




**Figure 4.3:** Graphical interface of the track children dashboard

***Description:*** This interface allows the parent to view their children's profiles and send real-time pickup notifications to the respective teachers. By notifying the school in advance, the parent ensures the child is ready upon arrival, effectively eliminating wait times and streamlining the dismissal process.

Interface for viewing today's scheduled activities

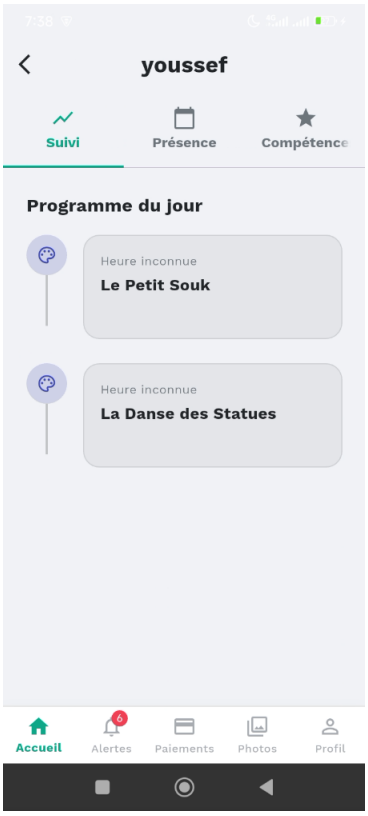



**Figure 4.4:** Graphical interface for viewing today's scheduled activities

***Description:*** This interface displays the list of activities scheduled for the selected child on the current day.

# Use Case name: Track Children

Interface for viewing daily attendance

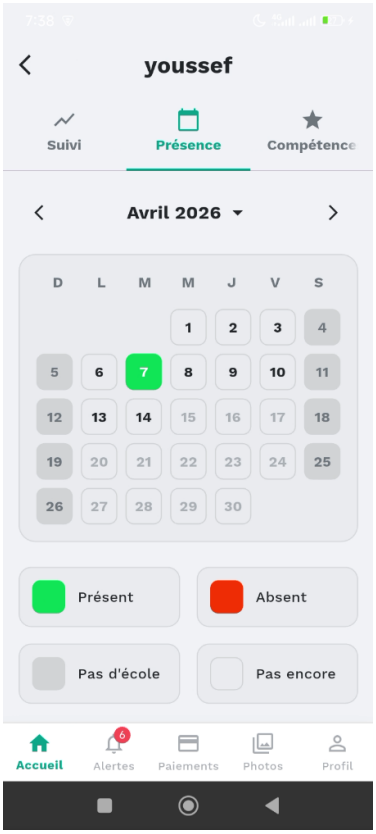



**Figure 4.5:** Graphical interface for viewing daily attendance

***Description:*** This interface allows the parent to review the daily attendance record of their child. It shows whether the child was present or absent on each school day, giving parents a clear and up-to-date overview of their child's attendance history.

Interface for viewing acquired competences




**Figure 4.6:** Graphical interface for viewing acquired competences

***Description:*** This interface presents the competence evaluation results for the selected child, grouped by activity. Each competence is marked as acquired or needing reinforcement, enabling parents to follow their child's pedagogical development over time.

## Use Case name: View Photos

Interface for browsing the photos




**Figure 4.7:** Graphical interface for browsing the photo gallery

***Description:*** This interface displays the photo gallery shared by the teacher for the selected child. Parents can browse all available photos and download them individually. Shared media is subject to an expiration period to ensure data privacy.

## Use Case name: Add Child

Interface for child re-registration

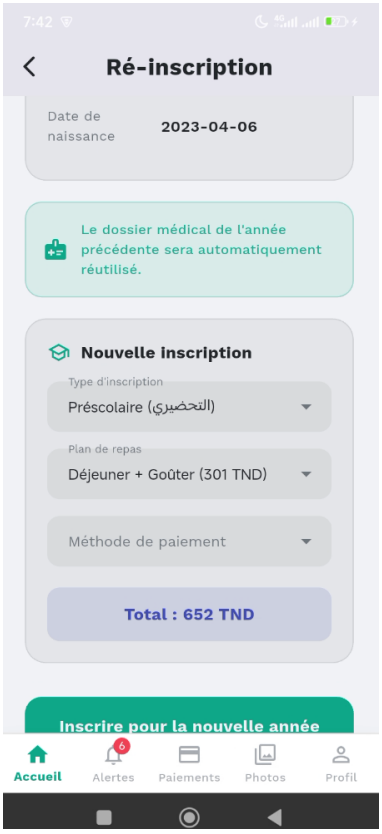



**Figure 4.8:** Graphical interface for child re-registration

***Description:*** This interface enables parents to efficiently re-register their existing children for the upcoming academic year. It allows the user to select new registration options, meal plans, and payment methods, while automatically calculating the total fees to ensure transparency before final submission.

## **Use Case name:** Review Payment History

Interface for viewing payment records

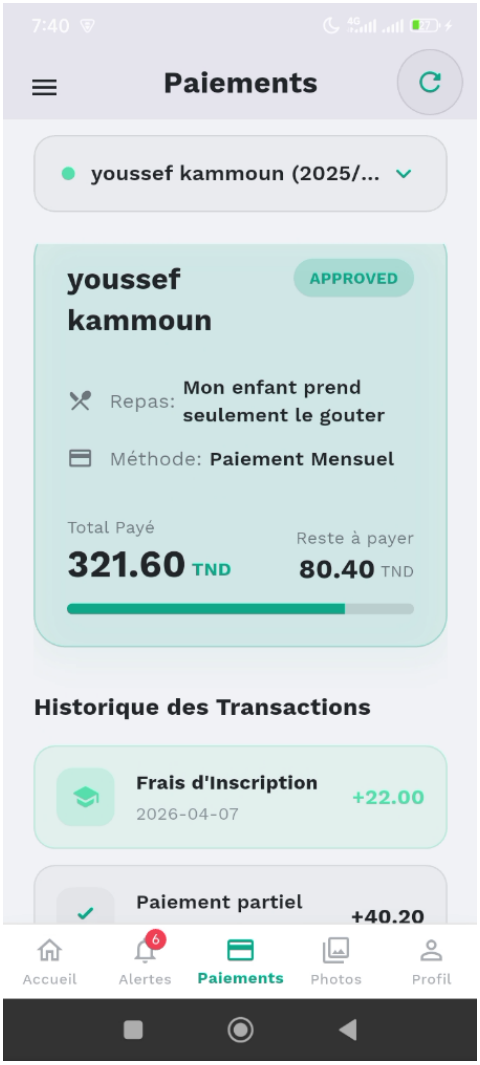



**Figure 4.9:** Graphical interface for viewing payment records

***Description:*** This interface provides parents with a comprehensive overview of the financial status for each child's registration. It displays specific registration details, including selected meal plans and payment methods, while tracking the total amount paid versus the remaining balance.

### 4.3.2 Teacher Use Cases

## Use Case name: View and Edit Activities

Interface for viewing class activities

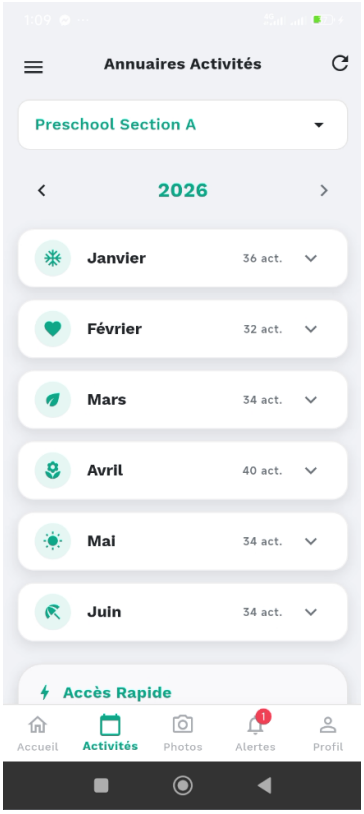



**Figure 4.10:** Graphical interface for viewing class activities

***Description:*** This interface gives teachers a complete overview of all activities planned for their assigned class throughout the current academic year. For each activity, the teacher can mark it as executed or not yet executed.

## Use Case name: Send Photos

Interface for photo sharing and upload

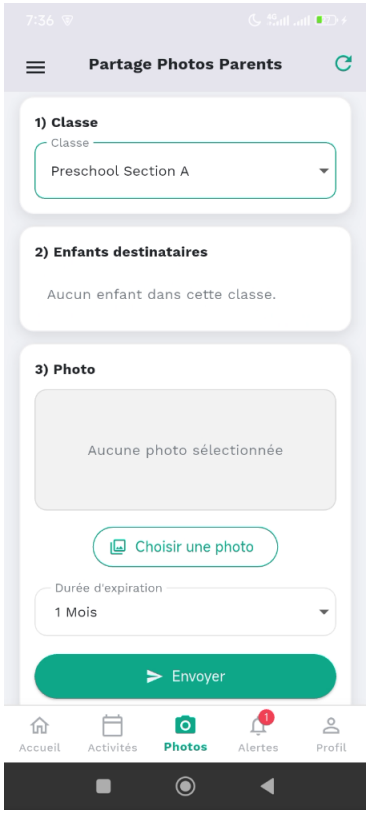



**Figure 4.11:** Graphical interface for photo sharing and upload

***Description:*** This interface allows authorized Teachers to share photos with parents by targeting specific classrooms. The process is organized into three steps: Class Selection, Recipient Identification, and Media Upload. The user can also set an expiration duration for the shared content to maintain data privacy.

**Use Case name:** Send Photos

```
return ListView(
  physics: const AlwaysScrollableScrollPhysics(),
  padding: const EdgeInsets.fromLTRB(16, 8, 16, 24),
  children: [
    SectionCard(
      title: '1) Classe',
      child: ClassSelector(controller: _controller),
    ), // SectionCard
    const SizedBox(height: 12),
    SectionCard(
      title: '2) Enfants destinataires',
      child: ChildrenSelector(controller: _controller),
    ), // SectionCard
    const SizedBox(height: 12),
    SectionCard(
      title: '3) Photo',
      child: PhotoUpload(controller: _controller),
    ), // SectionCard
    const SizedBox(height: 18),
    RecentShares(controller: _controller),
  ],
); // ListView
```

**Figure 4.12:** Graphical interface for completing the photo sharing workflow

## Use Case name: Manage Class Attendance

Interface for viewing assigned classes

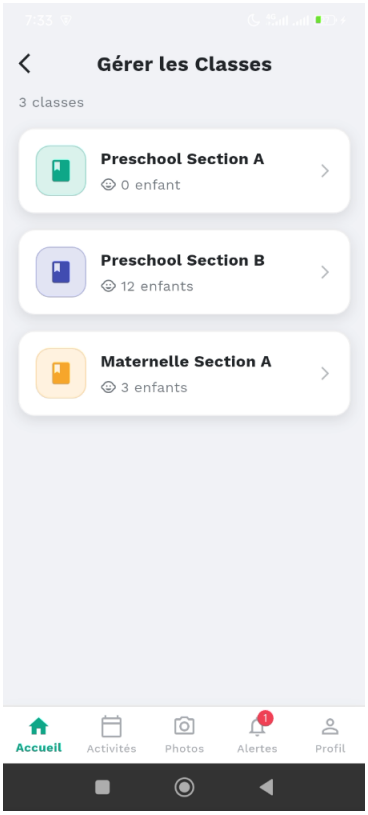



**Figure 4.13:** Graphical Interface for viewing assigned classes

***Description:*** This interface displays the details of the teacher's assigned class, including the class name and type. It serves as the main entry point for the teacher to access class-specific actions.

## Use Case name: Manage Class Attendance

Interface for recording class attendance

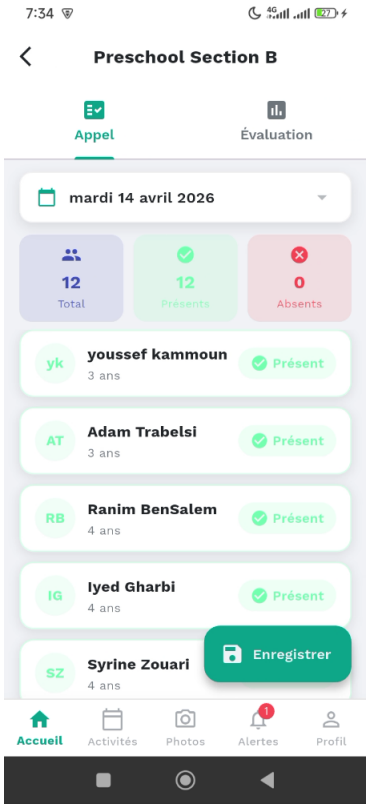



**Figure 4.14:** Graphical interface for recording class attendance

***Description:*** This interface allows the teacher to record the daily attendance for every child in the class. For each child, the teacher can mark their status as present, absent, or late. The system saves the attendance records in real time to ensure accurate tracking.

## **Use Case name:** Evaluate Child Competences

Interface for evaluating competences




**Figure 4.15:** Graphical interface for evaluating competences

***Description:*** This interface enables teachers to evaluate each child's mastery of the competences associated with a given activity. For each competence, the teacher assigns a grade status acquired or needs reinforcement which is then made visible to the parent.

### 4.3.3 Parent and Teacher Shared Use Cases

**Use Case name: Manage Profile**

Interface for viewing personal profile (Parent)




**Figure 4.16:** Graphical interface for viewing parent profile

***Description:*** This interface displays the parent's personal account information. It also lists their registered children along with each child's current registration status.

**Use Case name: Manage Profile**

Interface for viewing personal profile (Teacher)

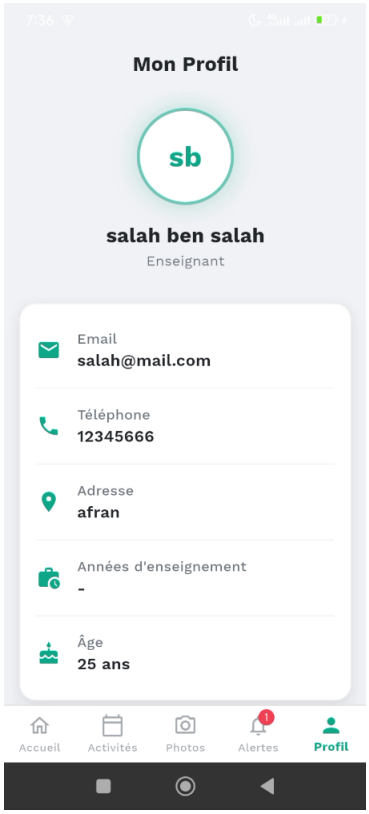



**Figure 4.17:** Graphical interface for viewing teacher profile

***Description:*** This interface displays the teacher's personal account information. It also shows the class they are currently assigned to, including the class name and the number of enrolled students.

## Use Case name: View Notifications

Interface for viewing notifications




**Figure 4.18:** Graphical interface for viewing notifications

***Description:*** This interface lists all notifications received by the user, such as pickup confirmations, registration updates, and system alerts. Unread notifications are visually highlighted, and tapping a notification marks it as read.

## Use Case name: Authenticate

Interface for mobile login

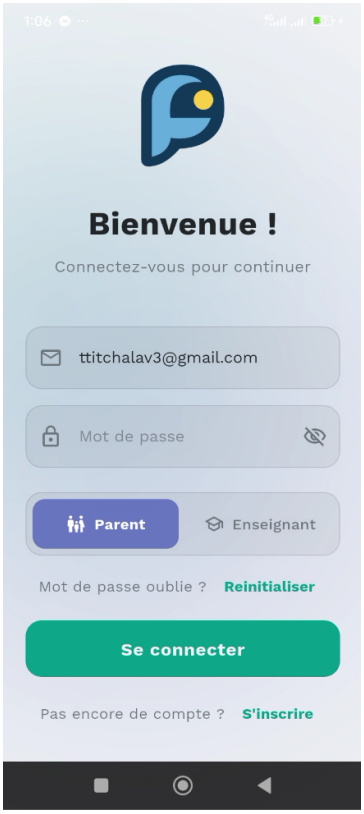



**Figure 4.19:** Graphical interface for mobile login

***Description:*** This interface is the entry point to the mobile application. The user enters their email address and password, selects their role (Parent or Teacher), and taps the login button. The system validates the credentials and redirects the user to their respective dashboard upon success.

### 4.3.4 Administrator Use Cases

**Use Case name:** Access Dashboard

Interface for Interface for viewing dashboard statistics

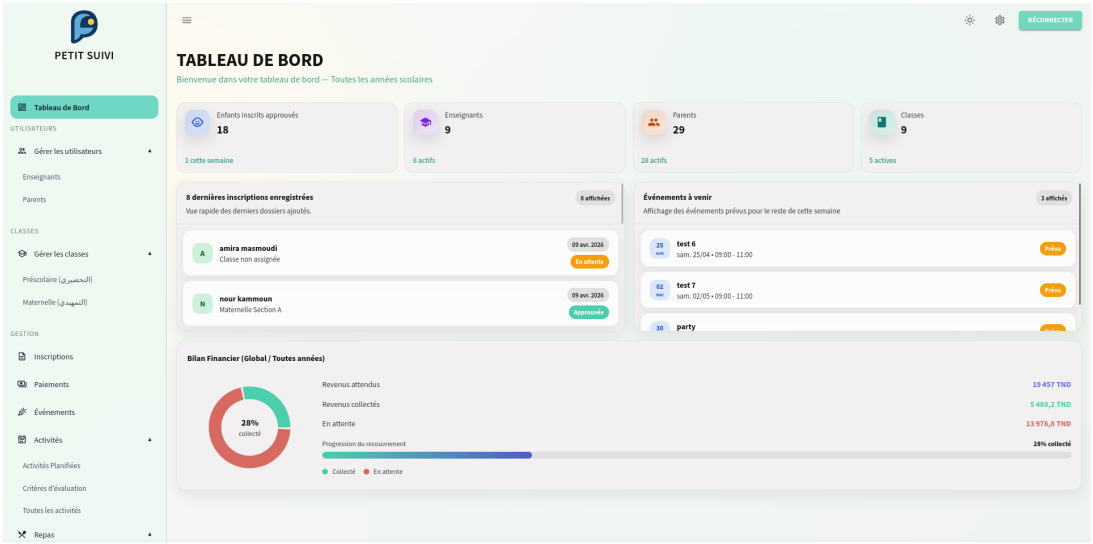



**Figure 4.20:** Graphical interface for viewing dashboard statistics

**Use Case name:** Manage Activities

Interface for Interface for activity creation

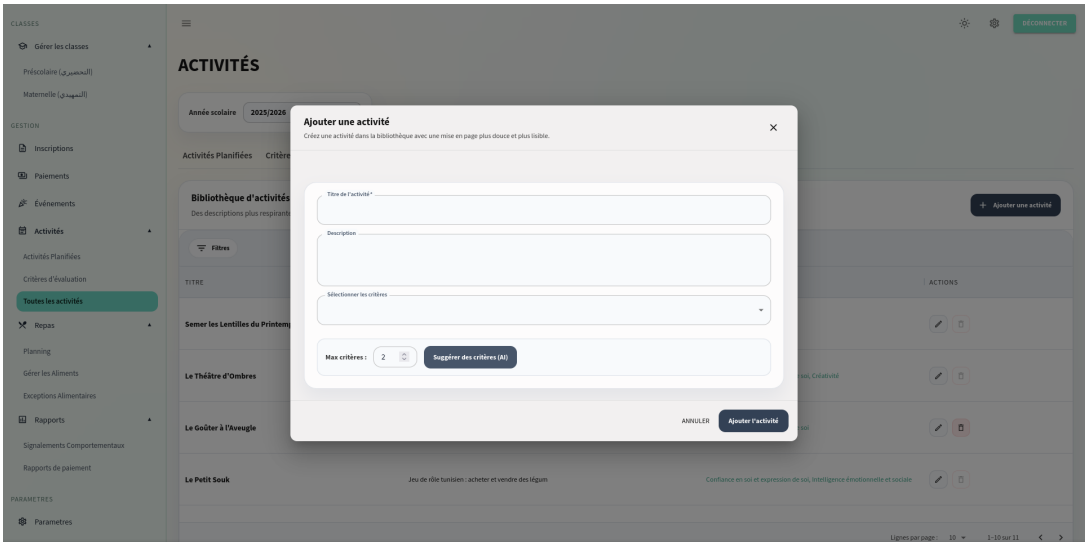



**Figure 4.21:** Graphical interface for activity creation

## **Use Case name:** Manage Payments
Interface for Interface for viewing payments

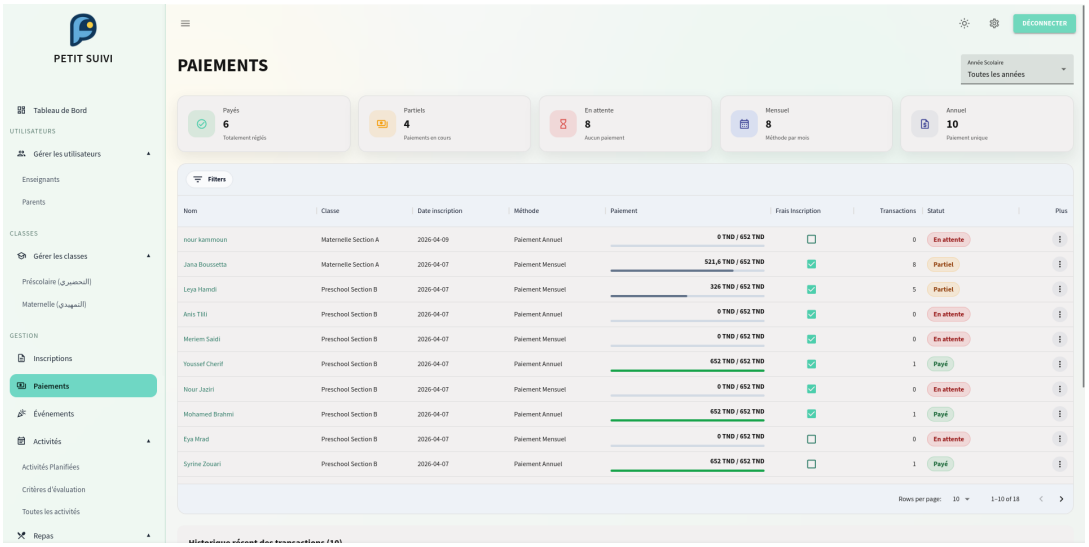



**Figure 4.22:** Graphical interface for viewing payments

## **Use Case name:** Manage Registrations
Interface for Interface for viewing registrations

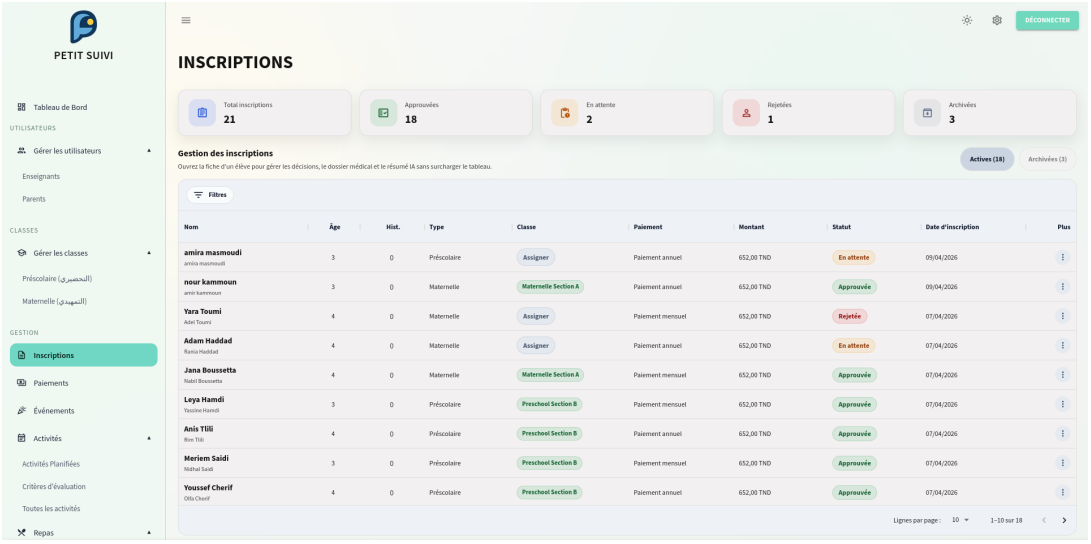



**Figure 4.23:** Graphical interface for viewing pending registrations

## **Use Case name:** Manage Registrations

Interface for approving registrations

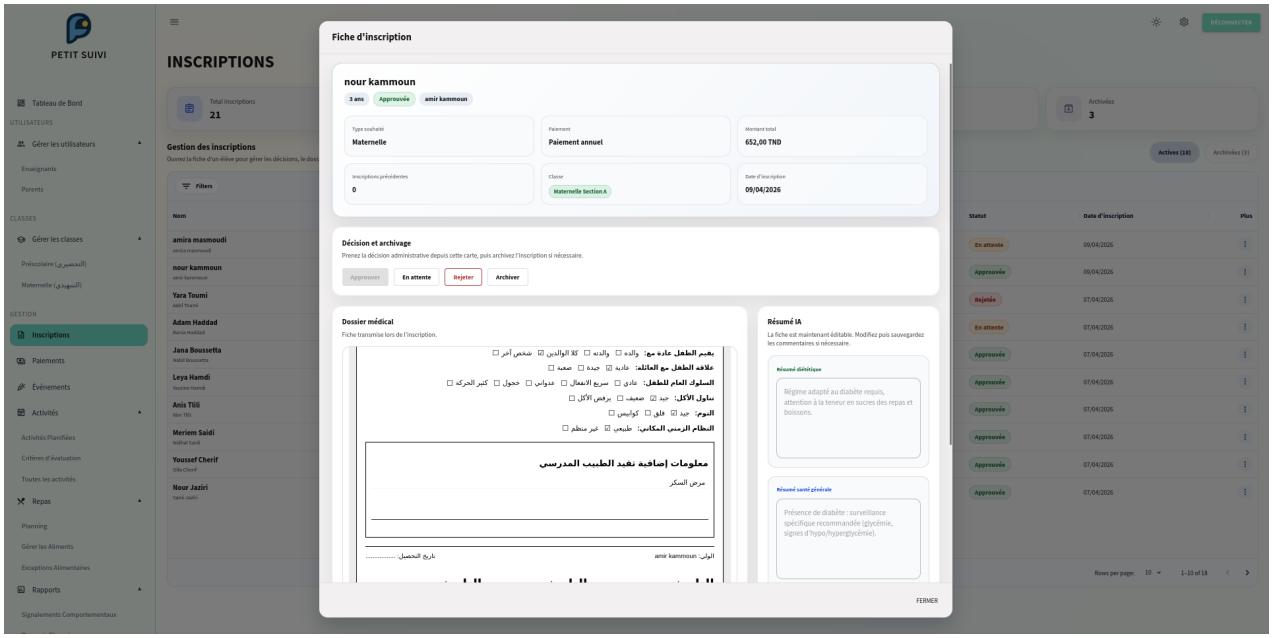



**Figure 4.24:** Graphical interface for approving registrations

## **Use Case name:** Manage Classes

Interface for creating classes

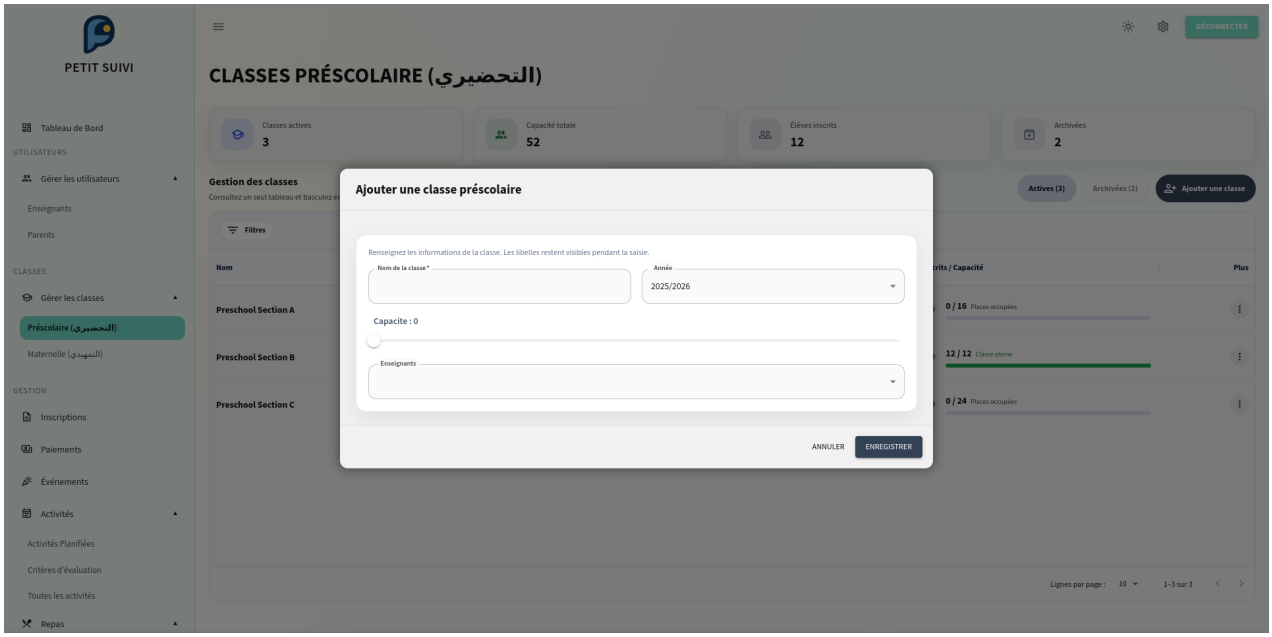



**Figure 4.25:** Graphical interface for creating classes

**Use Case name:** Manage Users

Interface for approving parent accounts

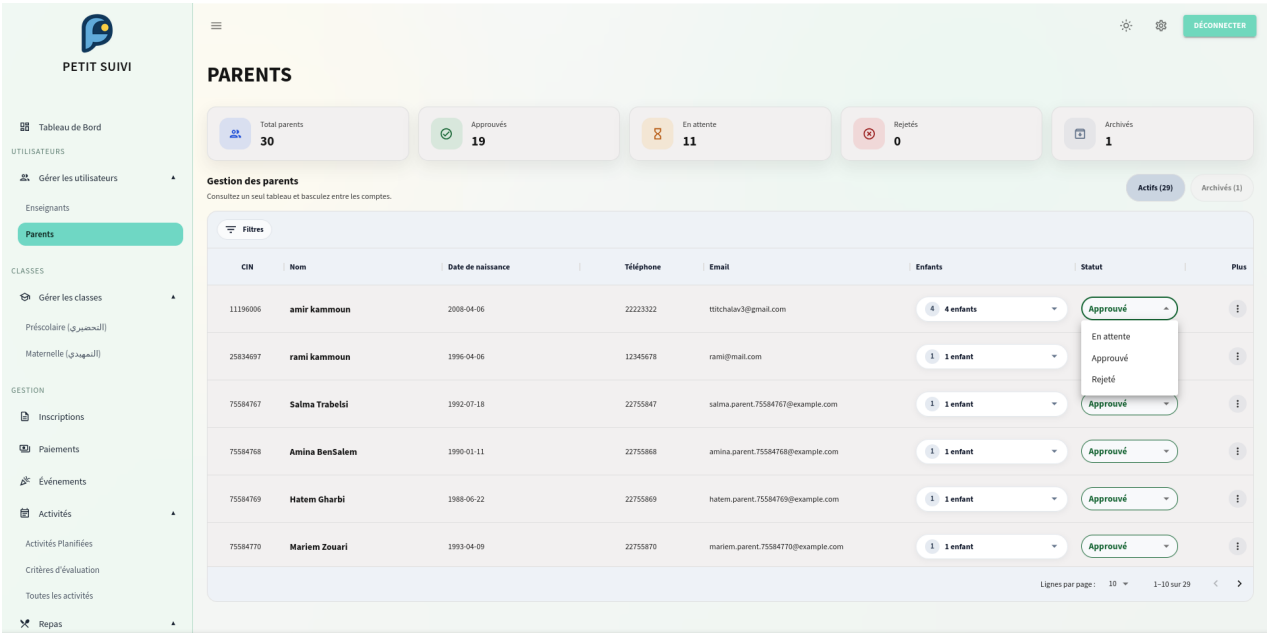



**Figure 4.26:** Graphical interface for approving parent accounts

**Use Case name:** Manage Users

Interface for viewing users

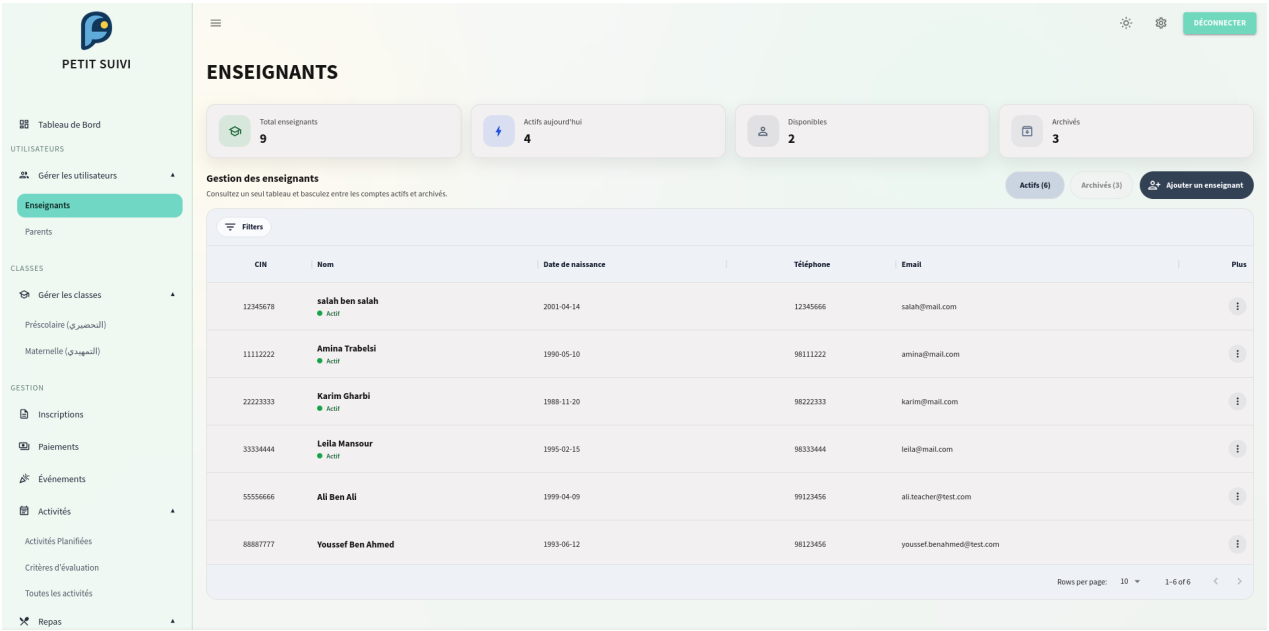



**Figure 4.27:** Graphical interface for viewing users

## **Use Case name:** Plan Meal Schedules

Interface for meal planning

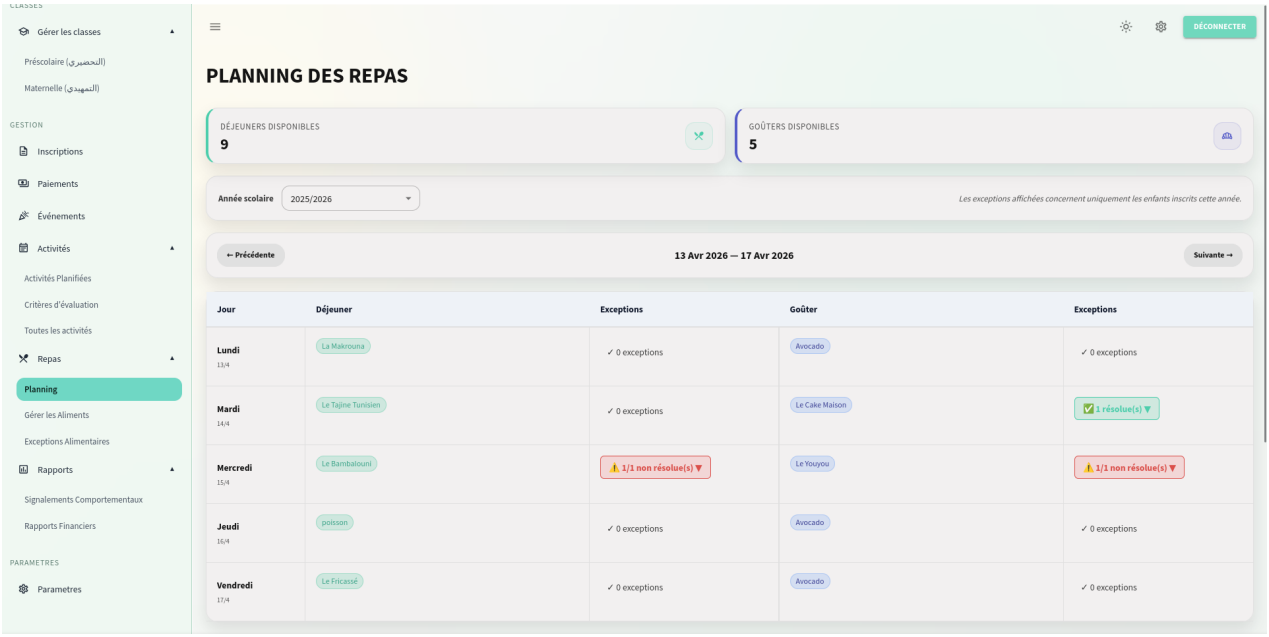



**Figure 4.28:** Graphical interface for meal planning

## **Use Case name:** Manage Planned Activities

Interface for viewing planned activities

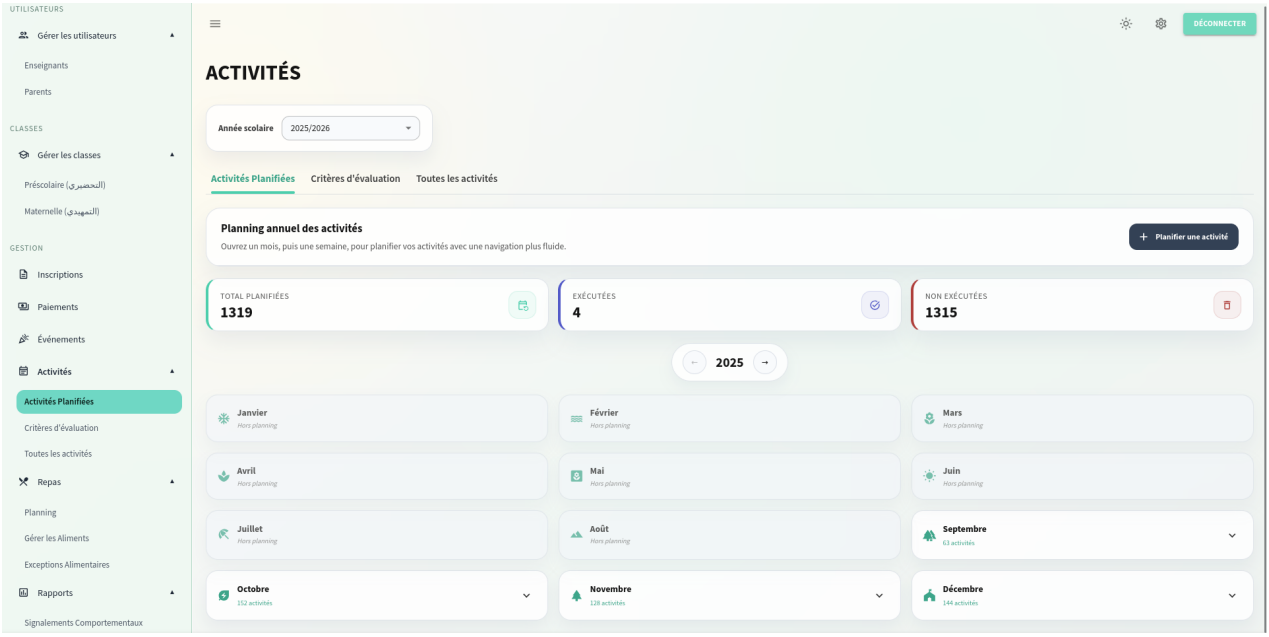



**Figure 4.29:** Graphical interface for viewing planned activities

***Description:*** **Data Integrity Protocol: Scheduling Constraints & Temporal Collision Detection**

**Core Identification**

- **API Route:** `POST /api/schoolyear/{id}/activities`
- **Controller:** `PlannedActivityController`
- **Function:** `store`

```php
$dayOfWeek = \Carbon\Carbon::parse($request->date)->dayOfWeek;
if ($dayOfWeek === \Carbon\Carbon::SATURDAY || $dayOfWeek === \Carbon\Carbon::SUNDAY) {
    return response()->json([
        'success' => false,
        'message' => 'Les activités ne peuvent être planifiées que du lundi au vendredi.'
    ], 422);
}
```

**Figure 4.30:** Implementation code: validate planned activity assignment rules

```php
// Early check for time overlaps if times are provided
if ($request->start_time && $request->end_time) {
    foreach ($request->class_ids as $classId) {
        $overlap = \Illuminate\Support\Facades\DB::table('Plandayactivity')
            ->join('Planday', 'Plandayactivity.PlandayID', '=', 'Planday.PlandayID')
            ->join('Class', 'Planday.ClassID', '=', 'Class.ClassID')
            ->join('Activity', 'Plandayactivity.ActivityID', '=', 'Activity.ActivityID')
            ->where('Planday.Date', $request->date)
            ->where('Planday.ClassID', $classId)
            ->whereNotNull('Plandayactivity.Starttime')
            ->whereNotNull('Plandayactivity.Endtime')
            ->where('Plandayactivity.Starttime', '<', $request->end_time)
            ->where('Plandayactivity.Endtime', '>', $request->start_time)
            ->first();

        if ($overlap) {
            return response()->json([
                'success' => false,
                'message' => 'L\'activité chevauche une autre activité ("' . $overlap->Title
            ], 409);
        }
    }
}
```

**Figure 4.31:** Implementation code: detect planned activity scheduling conflicts

# **Use Case name:** Configure Parameters

Interface for viewing system parameters

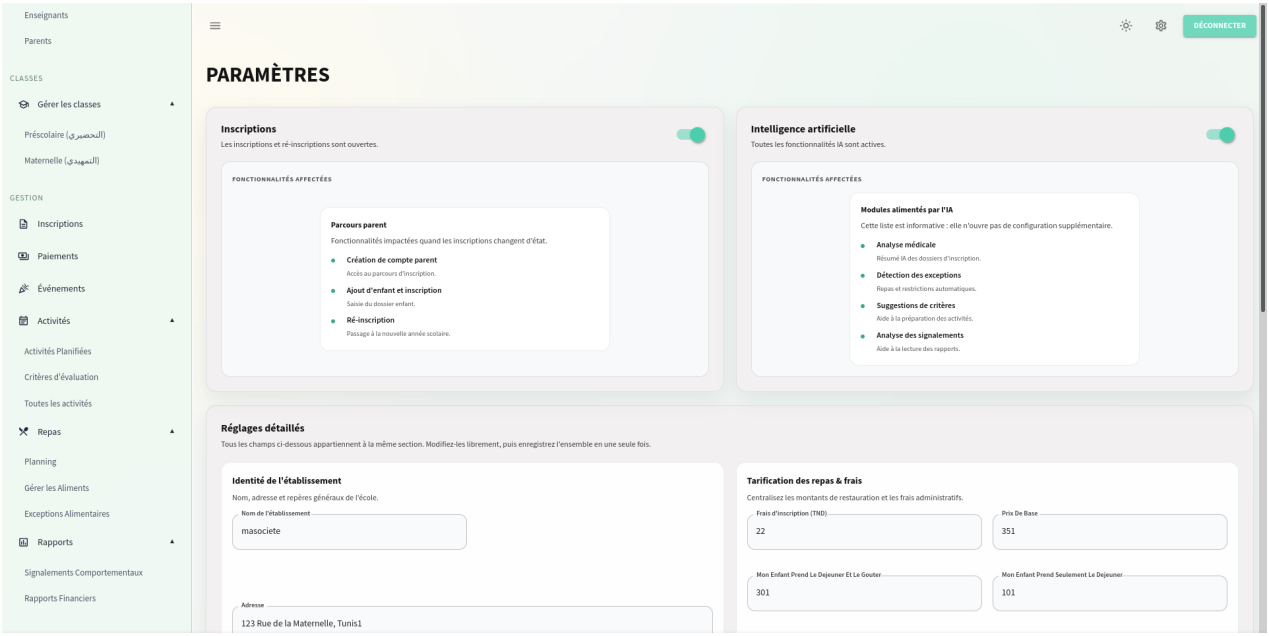



**Figure 4.32:** Graphical interface for viewing system parameters

# **Use Case name:** Configure Parameters

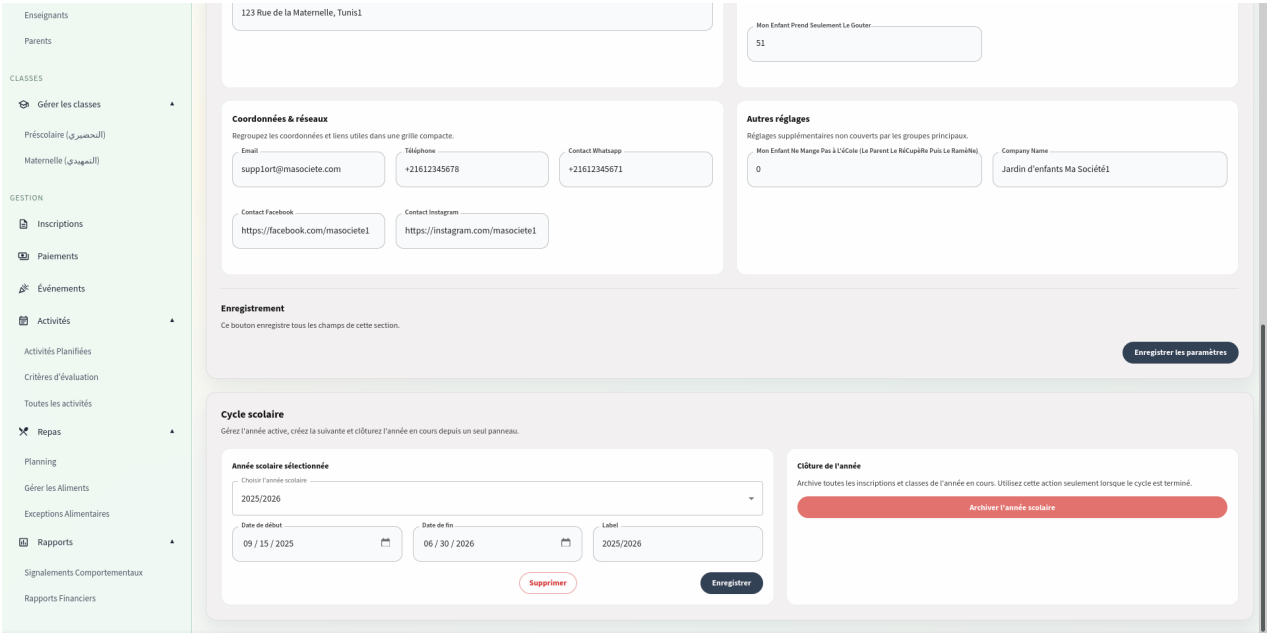



**Figure 4.33:** Graphical interface for viewing system parameters

# **Use Case name:** Generate Payment Reports

Interface for viewing financial reports

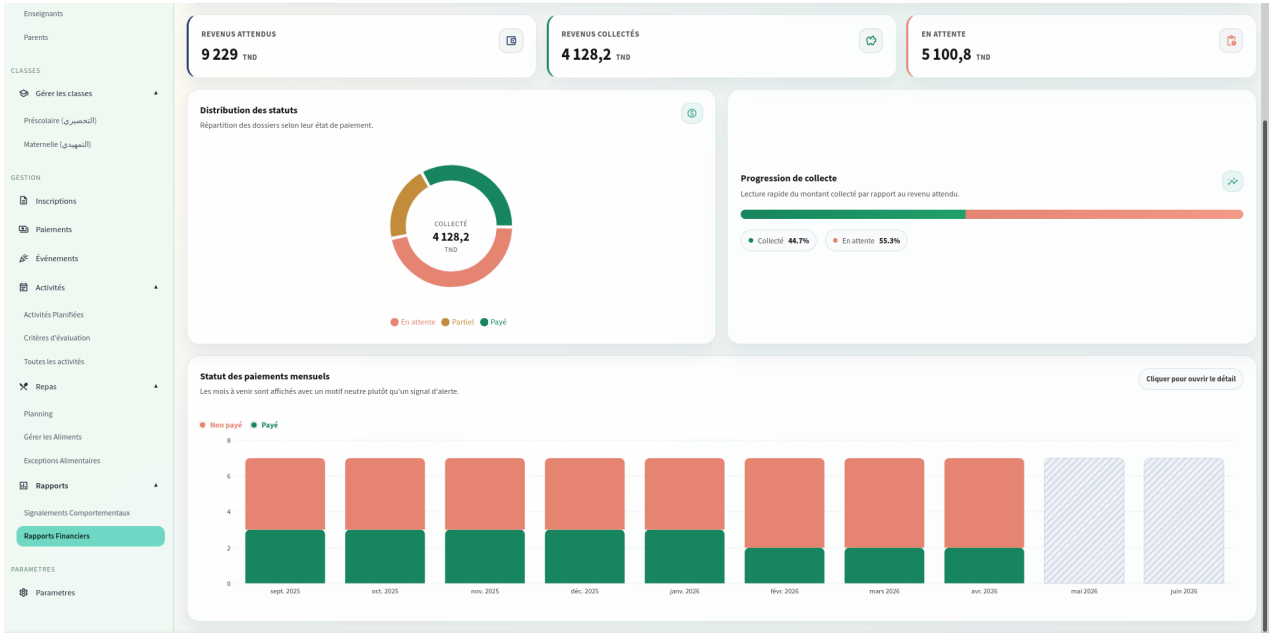



**Figure 4.34:** Graphical interface for viewing financial reports

# **Use Case name:** Generate Payment Reports

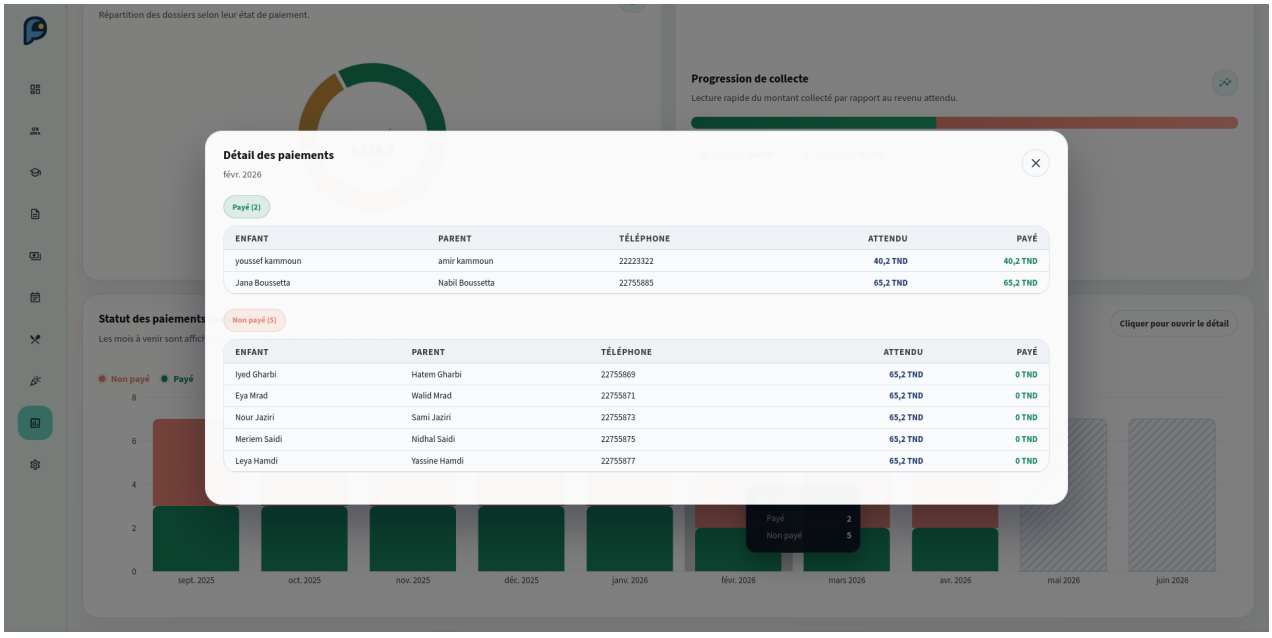



**Figure 4.35:** Graphical interface for viewing financial reports

### 4.3.5 System API Endpoints

| Section | Endpoint | Method |
|---|---|---|
| **Authentication & Profile** | | |
| Admin Login | /admin/login | POST |
| Admin Logout | /admin/logout | POST |
| Teacher Login | /api/login/teacher | POST |
| Parent Login | /api/login/parent | POST |
| Register User | /api/register | POST |
| Check Email Availability | /api/register/check-email | POST |
| Change Password | /api/password/change | POST |
| **User & Student Management** | | |
| Manage Teachers | /admin/teachers | GET, POST |
| Teacher Details | /admin/teachers/{cin} | PUT, DELETE |
| Archive Teacher | /admin/teachers/{cin}/toggle-archive | PATCH |
| Manage Parents | /admin/parents | GET |
| Parent Details | /admin/parents/{cin} | PUT, DELETE |
| Approve Parent | /admin/parents/{cin}/approval-status | PATCH |
| Manage Children | /api/parents/{cin}/children | GET, POST |
| View Child Photos | /api/children/{id}/photos | GET |
| View Child Payments | /api/parents/{cin}/payments | GET |
| **School Infrastructure** | | |
| Manage Classes | /admin/classes | GET, POST |
| Class Details | /admin/classes/{id} | PUT, DELETE |
| Manage Plannings | /admin/plannings | GET, POST |
| Archive School Year | /admin/plannings/archive-current-year | POST |
| Manage Parameters | /admin/parameters | GET, PUT |
| Fetch Signature | /admin/parameters/signature | GET |
| Upload Signature | /admin/parameters/signature | POST (Multipart) |
| Delete Signature | /admin/parameters/signature | DELETE |
| Teacher Class View | /api/teachers/{cin}/classes/by-planning | GET |

**Table 4.1:** System API Endpoints - Part 1

| Section | Endpoint | Method |
|---|---|---|
| **Activities & Pedagogical Tracking** | | |
| Manage Activities | /admin/activities | GET, POST, PUT, DELETE |
| Manage Criteria | /admin/criteria | GET, POST, DELETE |
| AI Activity Suggestions | /admin/ai/suggest-activity-criteria | POST |
| List Activities | /api/teachers/{cin}/classes/{id}/activities | GET |
| Update Activity Status | /api/teachers/{cin}/classes/{id}/activities/{aid}/status | PATCH |
| Bulk Evaluation | /api/evaluations/bulk | POST |
| View Evaluations | /api/children/{id}/evaluations/date/{date} | GET |
| **Nutrition & Food Management** | | |
| Manage Meals | /admin/meals | GET, POST, DELETE |
| Weekly Menu | /admin/meals/by-week/{date} | GET |
| Menu Planning | /admin/menu-plannings | GET, POST |
| Food Exceptions | /admin/food-exceptions | GET, POST, DELETE |
| **Communication & Notifications** | | |
| Report Incident | /api/signalements | POST |
| View Incidents | /api/children/{id}/signalements | GET |
| Mark Incident Read | /api/children/{id}/signalements/read | PATCH |
| Parent Notifications | /api/parents/notifications | GET |
| Pickup Request | /api/notifications/pickup | POST |
| Pickup Queue | /api/teacher/pickup-notifications | GET |
| Complete Pickup | /api/teacher/pickup-notifications/{id}/complete | PATCH |
| **Media & Assets** | | |
| Upload Photos | /api/teacher/photos | POST (Multipart) |
| Retrieve Photos | /api/teacher/photos | GET |

**Table 4.2:** System API Endpoints - Part 2

### 4.3.6 Database Triggers

To enforce critical business rules directly at the database level, several MySQL triggers were implemented. These triggers act as a safety net that validates data integrity before any insertion or modification is persisted, independently of the application layer.

#### Child Age Validation

This trigger ensures that only children aged between 2 years and 5 years and 4 months (24 to 64 months) can be registered in the system.

**Listing 4.1:** Trigger: Child Age Validation on Insert

```
CREATE TRIGGER trg_child_age_insert
BEFORE INSERT ON Child
FOR EACH ROW
BEGIN
    DECLARE age_months INT;
    IF NEW.Birthdate IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Child must have a birthdate.';
    END IF;
    SET age_months = TIMESTAMPDIFF(MONTH, NEW.Birthdate, CURDATE());
    IF age_months < 24 OR age_months > 64 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Child age must be between 2 and 5 years 4
           months.';
    END IF;
END;
```

#### Class Planning Validation

This trigger guarantees that every class is linked to a valid planning and class type, and automatically derives the academic year from the planning start date.

**Listing 4.2:** Trigger: Class Type and Planning Validation on Insert

```
CREATE TRIGGER trg_class_type_insert
BEFORE INSERT ON Class
FOR EACH ROW
BEGIN
    DECLARE v_startdate DATE DEFAULT NULL;
    IF NEW.FK_Class_TypeID IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Class must specify a Class Type.';
    END IF;
    IF NEW.FK_SchoolYearID IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Class must be linked to a School Year.';
    END IF;
    SELECT StartDate INTO v_startdate
    FROM SchoolYear
    WHERE SchoolYearID = NEW.FK_SchoolYearID
    LIMIT 1;
    IF v_startdate IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Associated School Year record not found.';
    ELSE
        SET NEW.Year = YEAR(v_startdate); END IF; END;
```

### Payment Integrity Check

These triggers ensure that every payment record is always associated with both a valid registration and a child, on both insert and update operations.

**Listing 4.3:** Triggers: Payment Null Check on Insert and Update

```
CREATE TRIGGER trg_payment_null_check_insert
BEFORE INSERT ON Payment
FOR EACH ROW
BEGIN
    IF NEW.FK_RegistrationID IS NULL OR NEW.FK_ChildID IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Payment␣must␣have␣both␣a␣RegistrationID␣and
           ␣a␣ChildID.';
    END IF;
END;
CREATE TRIGGER trg_payment_null_check_update
BEFORE UPDATE ON Payment
FOR EACH ROW
BEGIN
    IF NEW.FK_RegistrationID IS NULL OR NEW.FK_ChildID IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Payment␣must␣have␣both␣a␣RegistrationID␣and
           ␣a␣ChildID.';
    END IF;
END;
```

## 4.4 Conclusion

This chapter presented the practical implementation of the **Petit Suivi** platform by detailing the development environment, the selected technologies, and the deployment infrastructure used to deliver the solution. It also showed how several of the functional requirements identified in the previous chapters were concretely realized through the mobile application, the administrative dashboard, and the backend services.
In addition, the presentation of the API endpoints highlighted the technical mechanisms that ensure communication between the different system components and support the overall operation of the system. Overall, this chapter confirms that the proposed design was successfully transformed into a functional, secure, and deployable solution, which provides the basis for the final general conclusion of this work.

# General Conclusion

In this project, we successfully developed and deployed **Petit Suivi**, an integrated educational management application designed to facilitate and streamline the complex administrative tasks, pedagogical tracking, and communication processes within early childhood preschool institutions.

In the first chapter of this work, we presented the general project context, identified the limitations of existing solutions, and outlined our primary mission to bridge these gaps. The second chapter focused on identifying the different actors within our system and documenting their interactions through comprehensive use cases. This foundation allowed us to transition to the system design phase in Chapter 3, where we modeled the architecture and database logic. Finally, Chapter 4 constituted the core practical implementation of our project, detailing the hardware specifications and the presentation of the modern technologies utilized.

This end-of-studies project provided a profound opportunity to consolidate the theoretical knowledge acquired throughout our academic training by applying it to cutting-edge technologies. This experience allowed us to master precise UML modeling and dive deep into multi-platform development. Building the mobile application with the Flutter SDK presented distinct challenges in state management and cross-platform fluid UI design, but successfully overcoming these obstacles was highly rewarding. Furthermore, developing a highly reactive administrative dashboard using React.js and integrating Artificial Intelligence utilities via a Node.js microservice constituted a major part of our practical technical contribution.

Crucially, our work allowed us to discover the intricacies of modern distributed architectures. We learned how to build and integrate a robust Laravel API hosted on a remote server ecosystem, utilizing REST parameters and JSON formatting to seamlessly manage secure data communication between heterogeneous environments namely, the Dart-based mobile client, the JavaScript web dashboard, and the centralized MySQL database. We also practically explored modern DevOps practices, including continuous integration and deployment pipelines.

The daily involvement in this project alongside stakeholders was a unique occasion to cultivate our communication skills in a professional environment, ensuring the technical features perfectly matched real-world nursery workflows. It was a profoundly enriching experience on all fronts. Detailed screenshots explaining the practical operation and user interfaces of our application are presented in a separate manual annex of our project.

Although the primary objectives of the Petit Suivi project have been fully achieved, the application we developed is built with scalability in mind. In the future, this system could be enriched by more advanced features, such as deep machine-learning data analytics for predicting child behavioral trends, integrating local electronic payment gateways, or adding biometric attendance tracking, effectively rendering it even more interactive and comprehensive.

# Bibliography

[1] Mrs. Imen Ben Said, Mrs. Emna Feki. *Laravel Course*.

[2] Mrs. Mounira Ben Abdallah Kriaa, Mrs. Nahla Zaaboub Haddar, Mrs. Samira Bendbabis. *UML Course*.

[3] Mr. Fahmi Bargui, Mrs. Wahiba Dammak. *SQL Course (Triggers)*.

[4] Mrs. Khouloud Boukadi. *Flutter Course*.

[5] Mr. Slim Kallel. *REST API Course*.

# Webography


[W1] Laravel – Official Documentation.
https://laravel.com/docs/13.x

[W2] Flutter – Official Documentation.
https://docs.flutter.dev/

[W3] React – Official Documentation.
https://react.dev/learn

[W4] Sparx Systems – Enterprise Architect User Guide.
https://sparxsystems.com/enterprise_architect_user_guide/17.1/welcome/index.html

[W5] GitHub – Code Hosting and Version Control Platform.
https://github.com

[W6] Cloudflare – Developer Documentation.
https://developers.cloudflare.com/directory/

[W7] Docker – Official Guides and Documentation.
https://docs.docker.com/guides/

[W8] W3Schools – Web Development Reference.
https://www.w3schools.com/

[W9] MUI – React Component Library Documentation.
https://mui.com/

Petit Suivi – 2025/2026

Mahmoud Kammoun

**INTERNSHIP PERFORMED AT: Mega Pixel**

**Academic Year 2025/2026**